

Machine Learning (2)

Hunsik Shin

Department of Industrial and Data Engineering

{hunsik.shin}@hongik.ac.kr

Machine Learning Basics

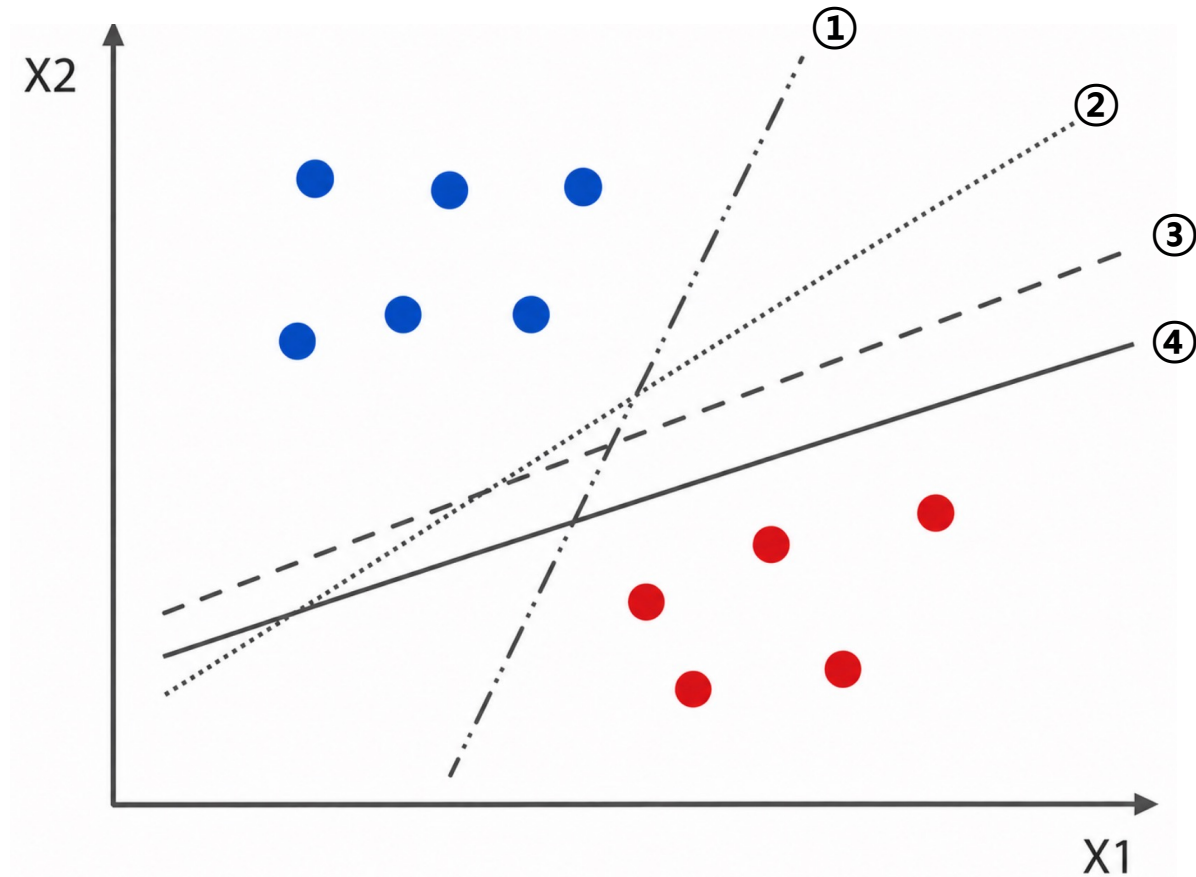
Support Vector Machine (SVM)

- Vapnik et al. (1992)의 statistical learning theory로 소개가 됨
- 주어진 데이터가 어느 카테고리에 속할지 판단하는 이진 선형 분류 모델
- 손필기 인식에 있어서 신경망보다 좋은 정확도를 보이면서 인기를 끌게 됨
- 1990년대 후반부터 대표적인 분류기로 널리 사용됨
- 현재는, 문자, 이미지, 음성 등의 데이터의 분류기로 폭넓게 사용되고 있음

Machine Learning Basics

Support Vector Machine (SVM)

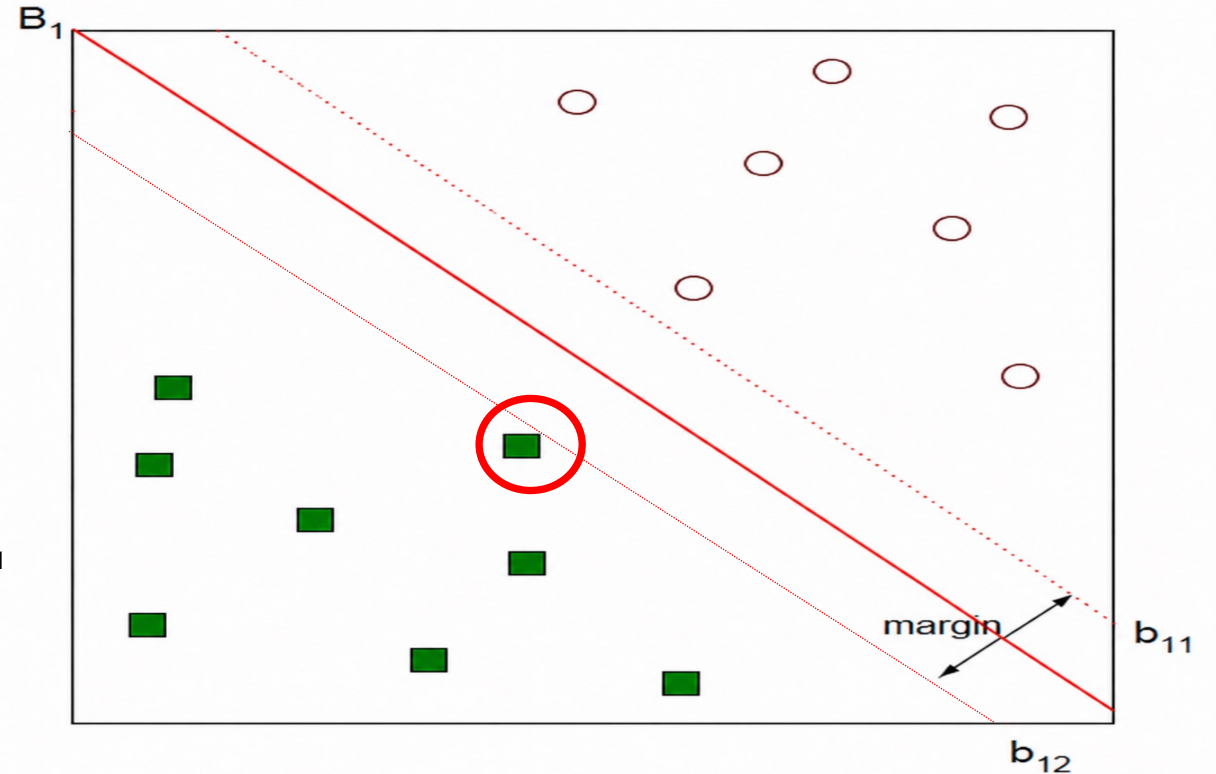
- 아래 그림에서 어떤 선이 가장 적절하게 두 데이터를 구분한 선일까?



Machine Learning Basics

Support Vector Machine (SVM)

- SVM의 마진은 결정경계와 가장 가까운 데이터 points* 사이의 거리
- SVM은 이 마진이 가장 커지는 결정경계를 찾아 두 클래스를 최대한 안정적으로 분리
- 마진이 클수록 새로운 데이터에 대한 분류 성능과 일반화 능력이 좋아질 가능성이 높음



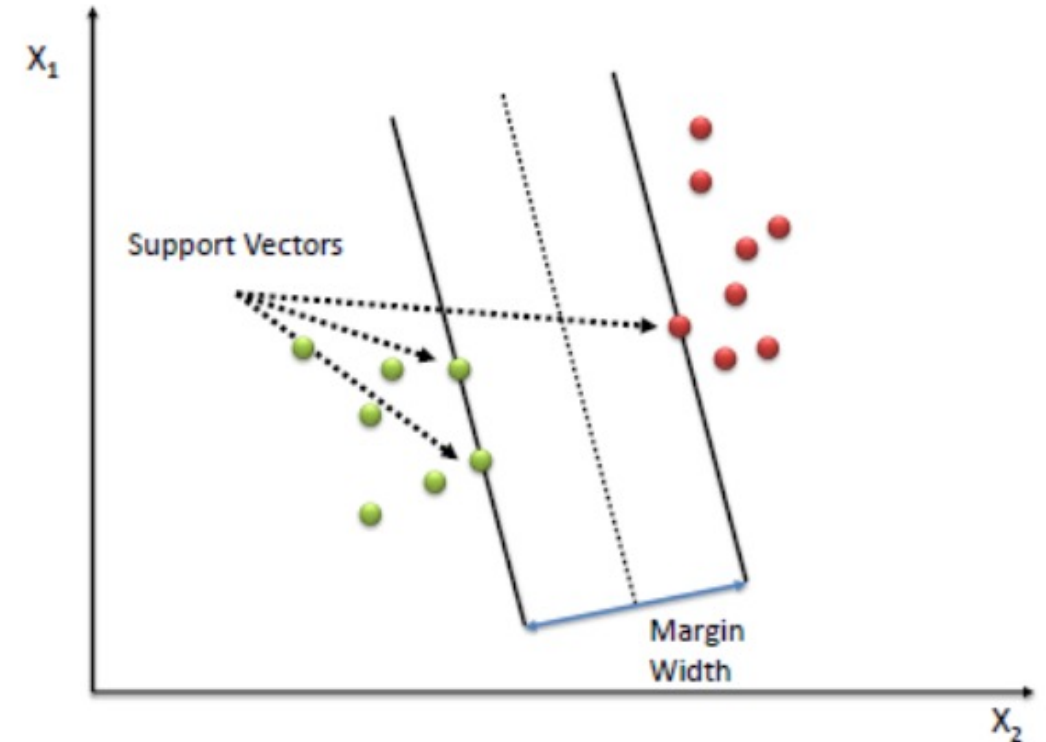
- 여러 개의 초평면(Hyperplane) 중 분류 경계면(Decision boundary)은 B_1
- b_{11} 과 b_{12} 사이의 거리를 **margin**이라고 함

* Support vectors

Machine Learning Basics

Support Vector Machine (SVM)

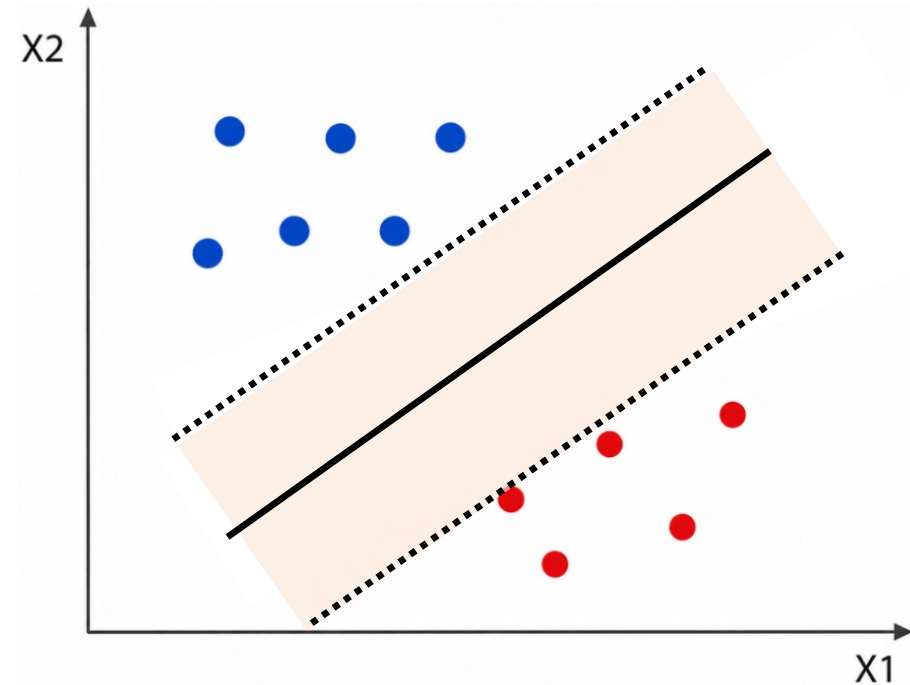
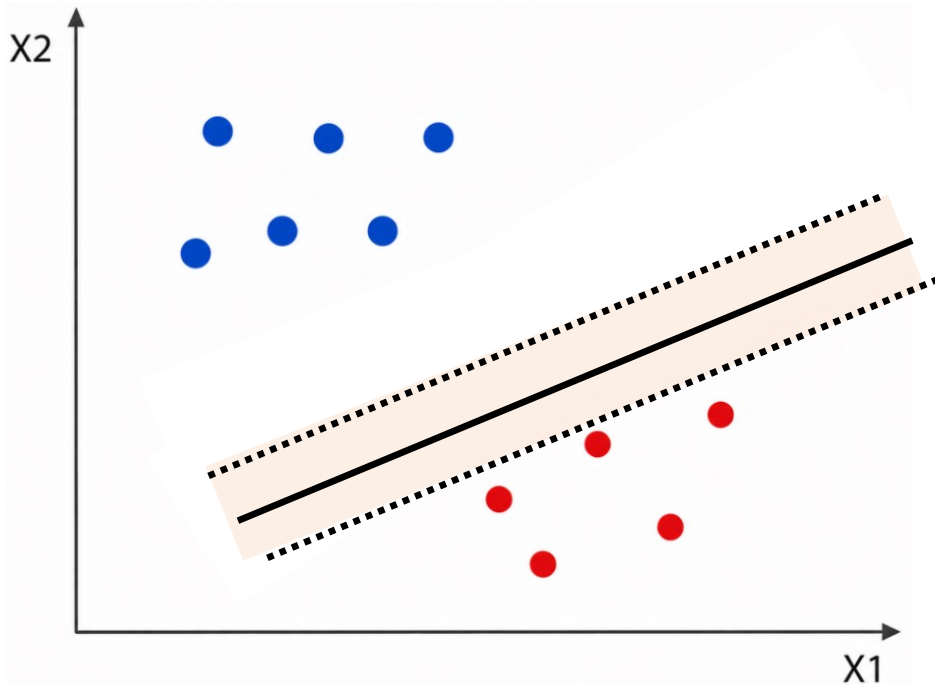
- Margin의 결정에 영향을 끼치는 관측치들을 서포트 벡터(Support vectors)라고 함.
- 서포트 벡터는 분류 경계면과 가장 가까운 점들임



Machine Learning Basics

Support Vector Machine (SVM)

- SVM은 margin이 최대화가 될 수 있는 초평면(오른쪽 그림의 optimal hyperplane, 판별경계)을 찾음

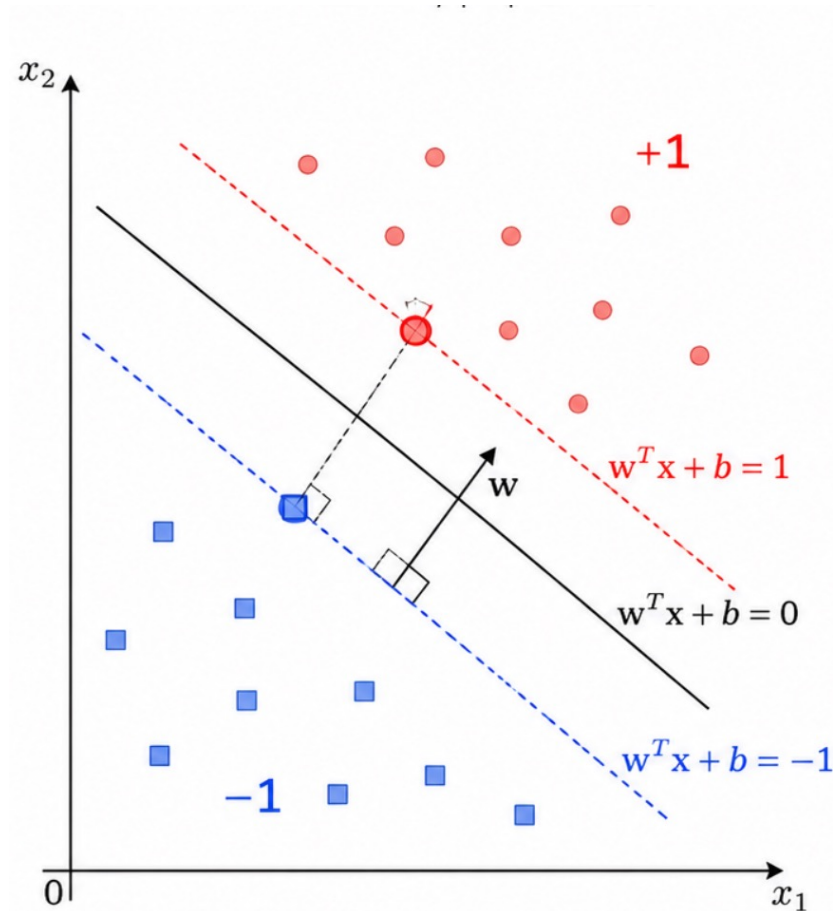


Machine Learning Basics

Support Vector Machine (SVM)

$$\max \frac{2}{\|w\|} = \min \frac{1}{2} \|w\|^2$$

$$\text{s.t. } y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n$$



1. 마진 경계와 결정경계 사이의 거리

결정경계: $w^T x + b = 0$

마진 경계: $w^T x + b = 1$ 또는 $w^T x + b = -1$

두 평행한 초평면 $w^T x + b = 0$ 과 $w^T x + b = 1$ 사이의 거리는 다음과 같습니다.

$$\text{거리} = \frac{|1 - 0|}{\|w\|} = \frac{1}{\|w\|}$$

(일반적으로 $w^T x + b = c_1$ 과 $w^T x + b = c_2$ 사이의 거리는 $\frac{|c_1 - c_2|}{\|w\|}$)

2. 마진의 전체 너비

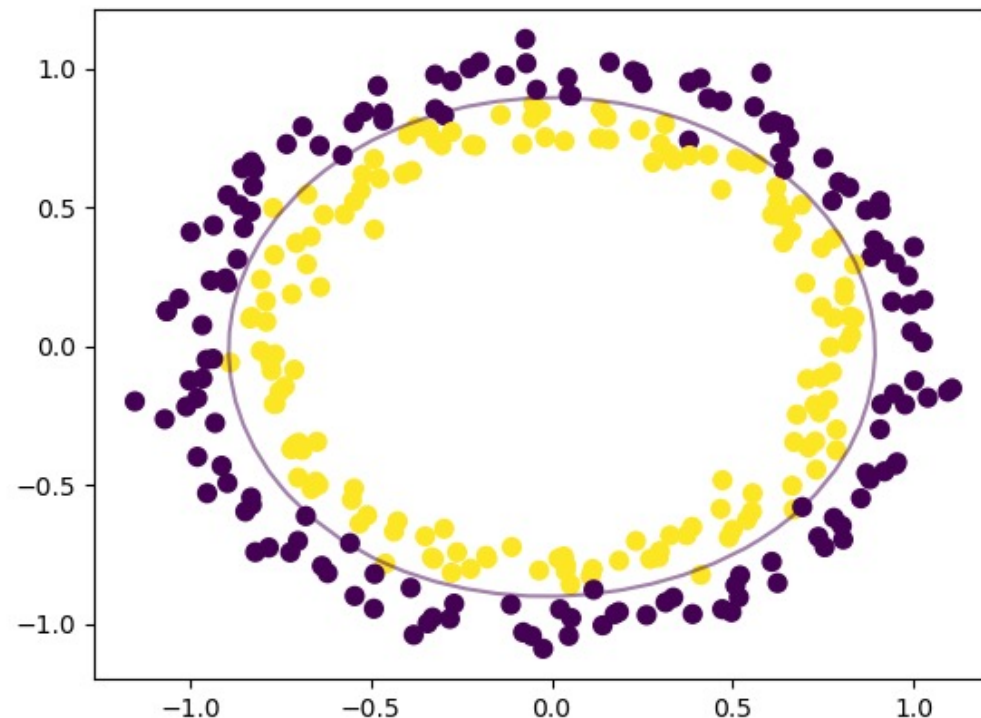
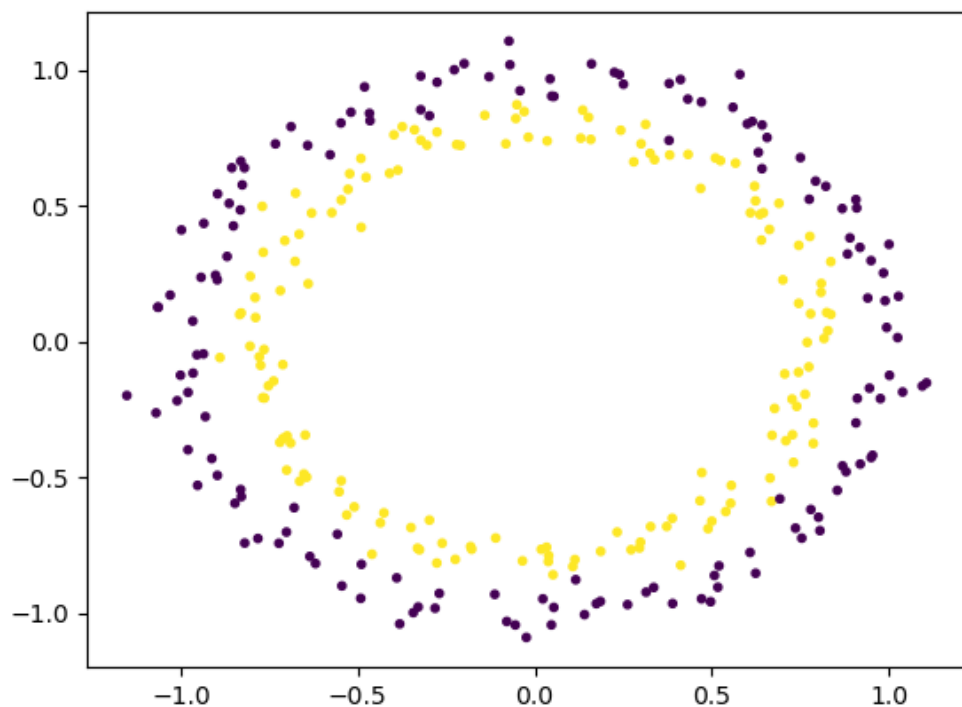
마진은 위쪽 경계($w^T x + b = 1$)와

아래쪽 경계($w^T x + b = -1$) 사이의 거리이므로,

$$\text{마진} = \frac{1}{\|w\|} + \frac{1}{\|w\|} = \frac{2}{\|w\|}$$

Machine Learning Basics

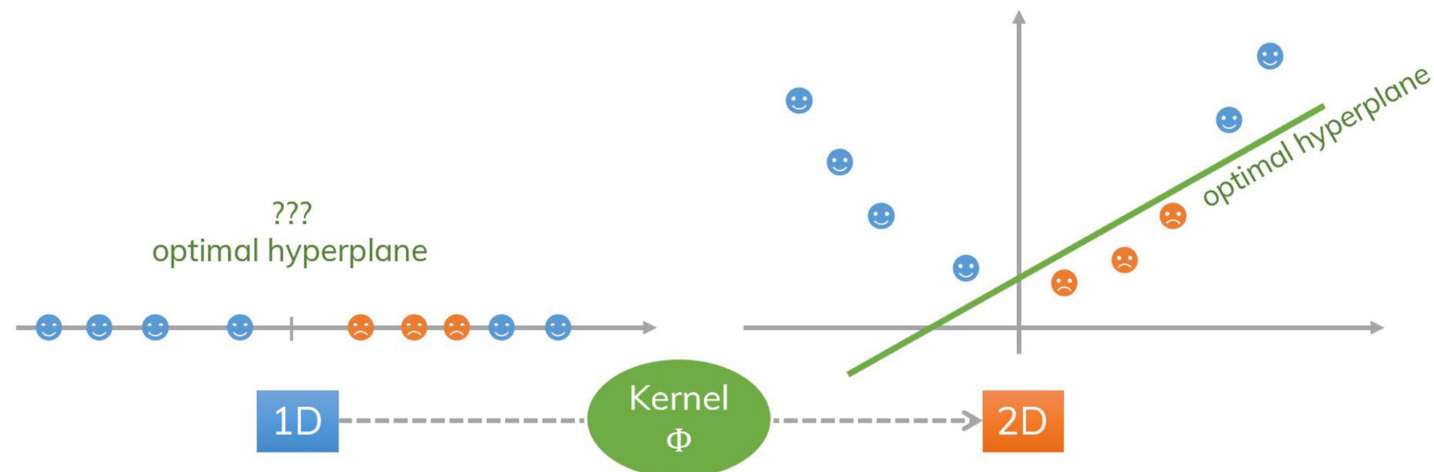
Support Vector Machine (SVM): 선형 vs 비선형



Machine Learning Basics

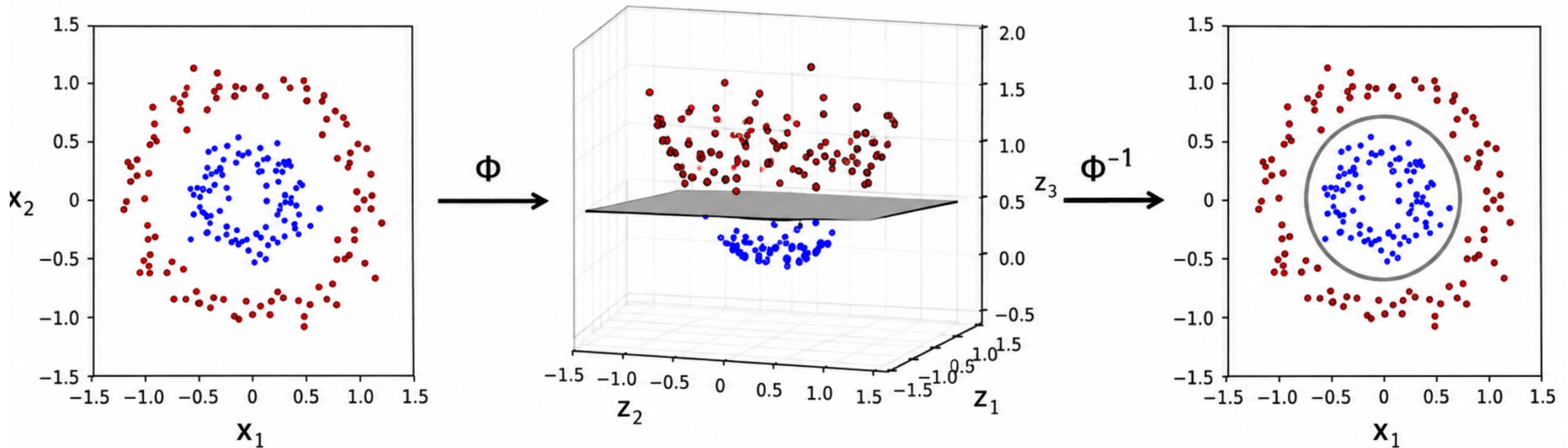
비선형 Support Vector Machine (SVM) → Kernel 함수의 이용

- 낮은 차원(2차원)의 비선형 분류경계를 가진 데이터들이 있다면, 이 입력벡터들을 고차원의 특정 공간으로 사상(mapping)시킨 후 선형 모델로 변환함
- 입력 벡터들을 kernel 함수를 통해 고차원 공간에 매핑한 후 초평면을 찾을 수 있음
- 즉, 데이터의 위상을 변화하여 초평면에 의해 분류가 가능하도록 함
 - 구별이 가능한 방향으로 사상(mapping)을 시키면 새로운 공간 영역으로 변환
 - 새로운 공간 영역에서는 초평면에 의해 분류가 가능해짐



Machine Learning Basics

비선형 Support Vector Machine (SVM) → Kernel 함수의 이용



출처: https://miro.medium.com/v2/resize:fit:400/0*Zpid3VFKDuD8T56W.png

Machine Learning Basics

Support Vector Machine (SVM) 장점 및 단점

장점

- 범주형 데이터나 수치형 데이터의 예측 문제에 사용.
- 알고리즘 추론이 매우 빠름.
- 고차원의 데이터에 대해서도 잘 작동함.

단점

- 데이터 스케일(양)이 증가할 경우, 계산량이 빠르게 높아짐. (학습 시간 오래 걸림)
- Outlier에 민감한 편임. Outlier가 많으면 성능이 많이 떨어짐.
- 분류 집단수가 증가할수록 복잡도가 지수적으로 증가함.
- 커널 함수 선택이 명확하지 않음.

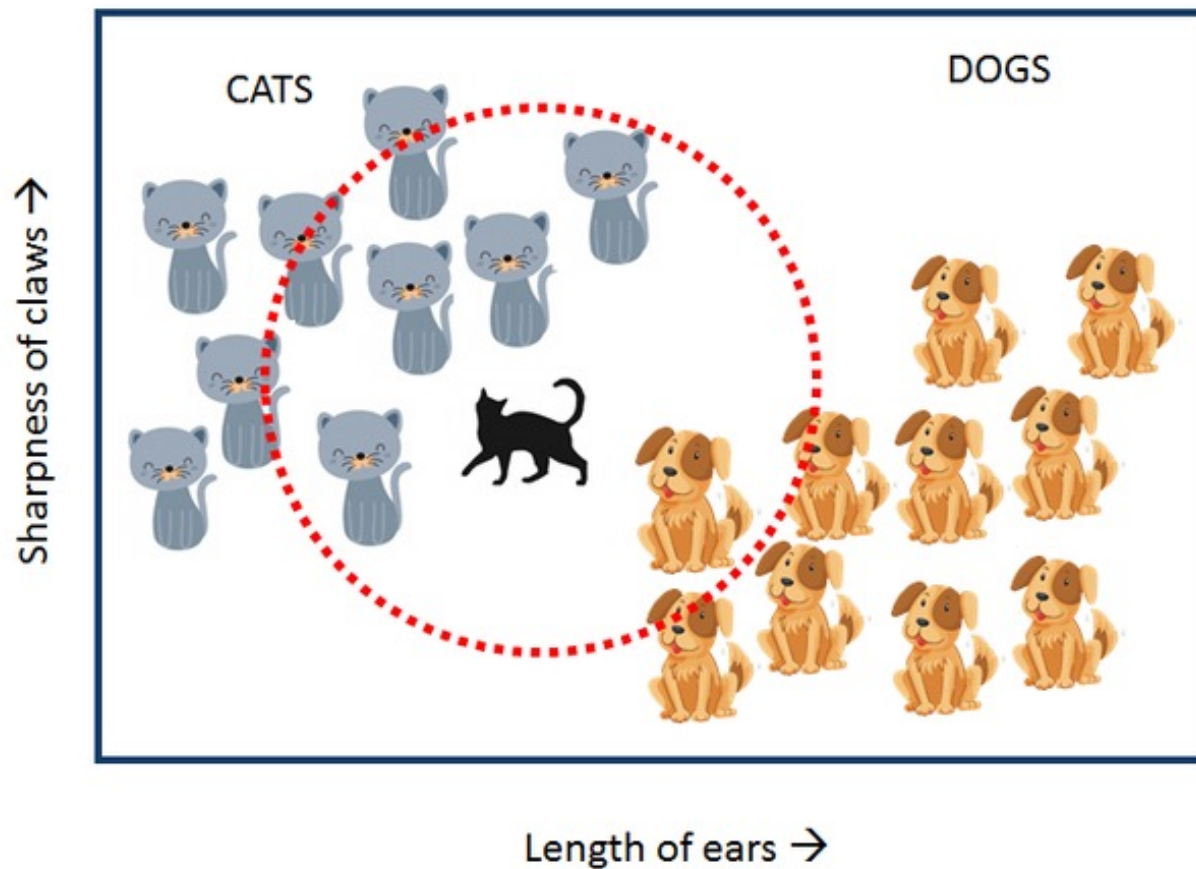
Machine Learning Basics

k-nearest neighbor (k-NN)

- 지도학습 알고리즘
- 기초적이지만 중요한 분류기법(classification) 중 하나
- 패턴 인식(pattern recognition), 데이터마이닝, 침입 탐지 시스템(intrusion detection system) 등에 사용
- 주어진 prior data (training data라고도 불림)을 이용하여 특징에 따라 여러 그룹으로 분류.

Machine Learning Basics

k-nearest neighbor (k-NN)



출처: <https://www.quora.com/What-type-of-problem-does-a-KNN-algorithm-solve-in-the-real-world>

Machine Learning Basics

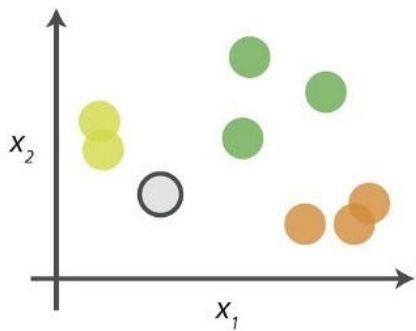
k-nearest neighbor (k-NN)

- kNN 분류 모형은 새로운 데이터(설명변수값)에 대해 이와 가장 유사한 (거리가 가까운) k-개의 과거 자료(설명변수값)의 결과(반응변수: 집단)를 이용하여 다수결(majority vote)로 분류하는 알고리즘.
- 만약 다수결 투표결과 동률이면 랜덤하게 할당하거나 가장 가까운 이웃에 따라 할당하는 방식을 택함
- 과거 자료를 이용하여 미리 분류모형을 수립하는 것이 아니라, 과거 데이터를 저장만 해두고 필요시 비교를 수행하는 방식임. → Lazy 알고리즘
- k값의 선택에 따라 새로운 데이터에 대한 분류결과가 달라짐에 유의
 - 일반적으로 k값은 홀수를 사용 (짝수일 때 동점인 경우 문제 발생)

Machine Learning Basics

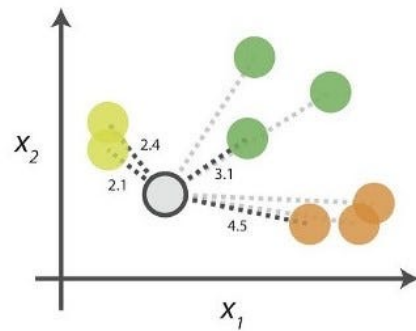
k-nearest neighbor (k-NN)

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point Distance			
		2.1	→ 1st NN
		2.4	→ 2nd NN
		3.1	→ 3rd NN
		4.5	→ 4th NN

Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

3. Vote on labels

Class	# of votes	
	2	→ Class  wins the vote! Point  is therefore predicted to be of class  .
	1	
	1	

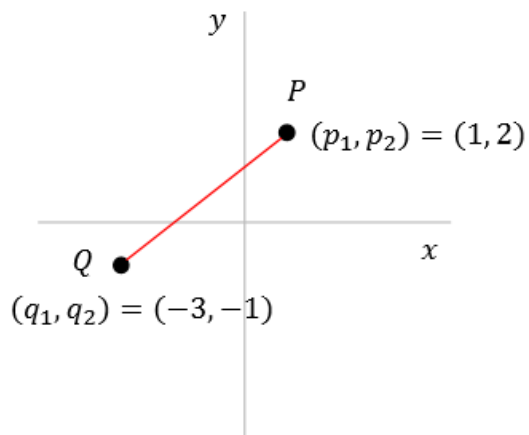
Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the k=3 nearest neighbours.

Machine Learning Basics

k-nearest neighbor (k-NN)

- 거리

- Euclidean distance
- Manhattan distance
- Minkowski distance
- Correlation distance



▪ Manhattan distance

$$d = \sum_{i=1}^n |p_i - q_i|$$

$$d = |1 - (-3)| + |2 - (-1)| \\ = 7$$

▪ Euclidean distance

$$d = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

$$d = \sqrt{(1 - (-3))^2 + (2 - (-1))^2} \\ = 5$$

▪ Minkowski distance

$$d = \sqrt[m]{\sum_{i=1}^n |p_i - q_i|^m}$$

$$d = \sqrt[3]{|1 - (-3)|^3 + |2 - (-1)|^3} \\ = 4.498$$

Machine Learning Basics

k-nearest neighbor (k-NN)

- kNN는 거리 기반으로 작동하는 알고리즘이므로 데이터에 대한 scale에 영향을 받음.
- 변수의 scale에 따라 상대적으로 가깝지만 거리가 훨씬 먼 것처럼 계산될 수 있기 때문에 scaling 작업 필요
- Scaling 작업을 안하면, 예를 들어, 소득에서 10원 차이나는 경우와 나이가 10살 차이나는 경우를 동일하게 처리하게 됨.

Machine Learning Basics

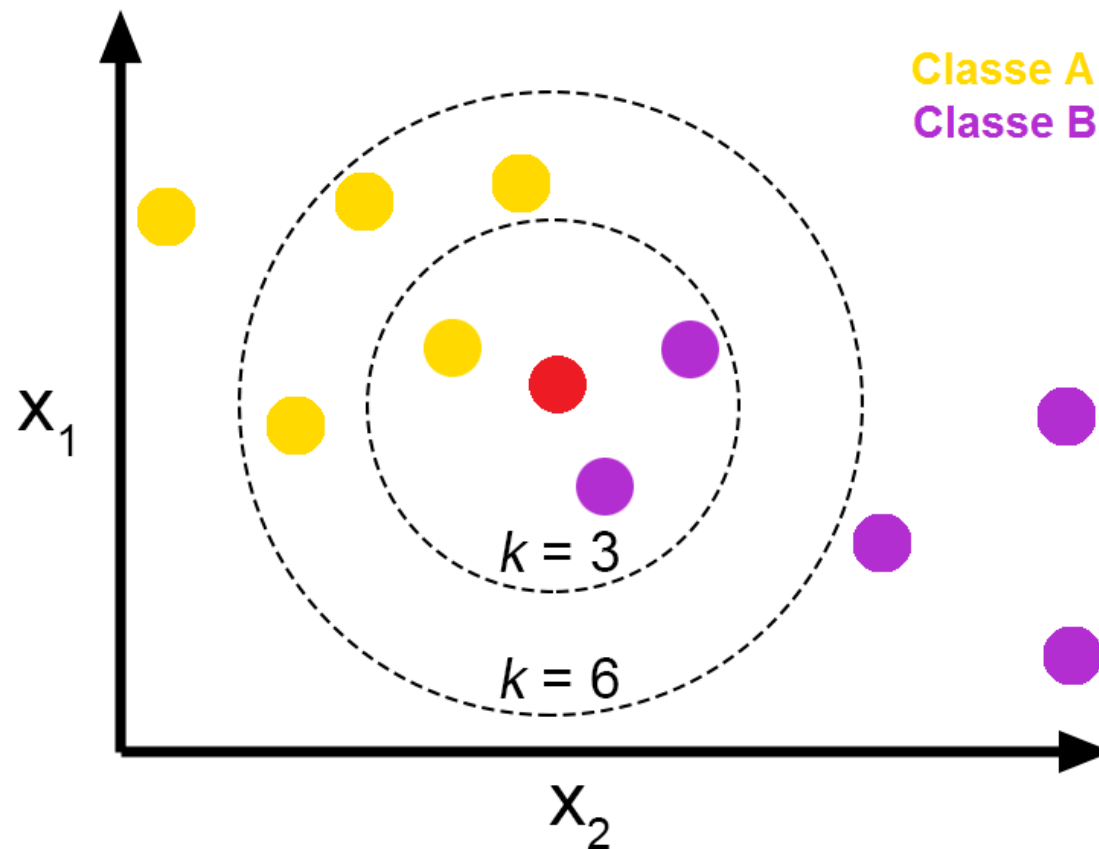
k-nearest neighbor (k-NN)

k = 3 이면 작은 원을 기준으로

- 노란색은 1개로, 확률 = 1/3
 - 보라색은 2개로, 확률 = 2/3
- ● 은 보라색으로 분류됨

k = 6 이면 큰 원을 기준으로

- 노란색은 4개로, 확률 = 4/6
 - 보라색은 2개로, 확률 = 2/6
- ● 은 노란색으로 분류됨



Machine Learning Basics

k-nearest neighbor (k-NN)

- kNN은 반응변수가 범주형인 경우에는 분류(classification)의 목적으로, 반응변수가 연속형인 경우에는 회귀(regression)의 목적으로 사용될 수 있음.
- kNN은 기계학습 분야에서 가장 단순한 알고리즘. 이 알고리즘은 학습데이터 셋과 데이터 간의 '거리' 만을 사용하여 분류하기 때문에 분류를 위한 수식이 필요 없음.
- 또한, 모든 계산이 이루어진 후에 분류가 이루어지는 특징으로 인해 사례-기반 학습(instance-based learning) 또는 게으른 학습(lazy learning)의 한 유형으로 볼 수 있음.
- 다른 머신러닝과 다르게 학습을 해서 새로운 데이터가 들어왔을 때 분류를 하는 것이 아니고, 현재 가지고 있는 데이터 전체를 활용해서 그 데이터를 기반으로 의사결정을 진행함. 따라서 별도의 학습을 하진 않지만 데이터를 항상 가지고 있어야 하고 매번 모든 데이터를 사용해야 하기 때문에 오래 걸린다는 문제가 있을 수 있음.

Machine Learning Basics

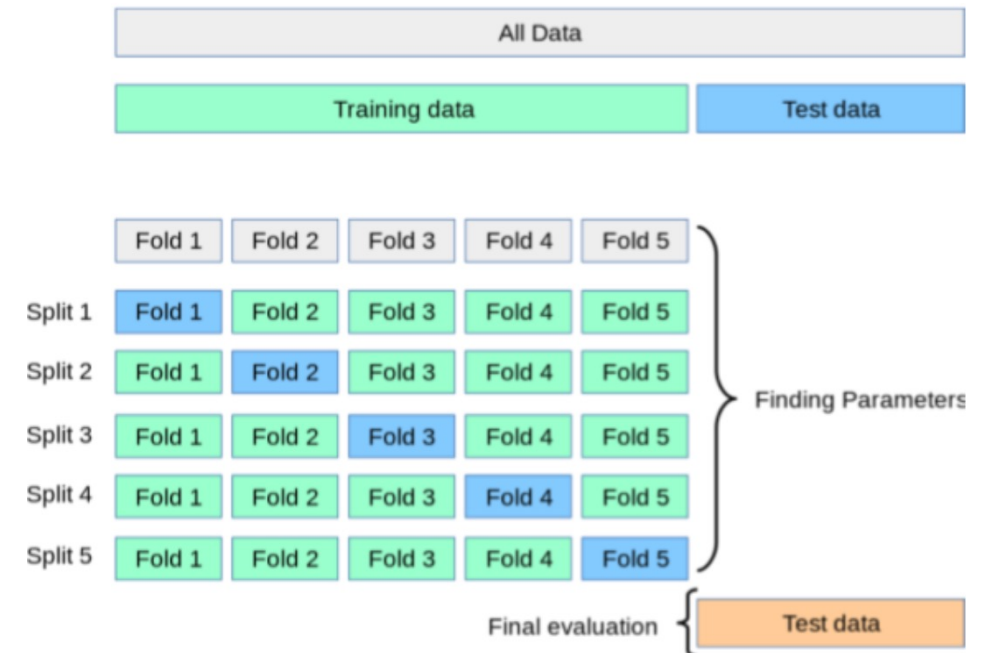
k-nearest neighbor (k-NN)

kNN에서는 이웃의 수 k 를 잘 정해주어야 함.

- k 값이 작으면 이상치의 영향을 크게 받게 되어 overfitting의 우려가 있음
- k 값이 크면 미세한 경계부분을 잘못 분류해 underfitting의 우려가 있음

적절한 k 값 찾기

- Cross validation으로 최적의 k 값을 구함.



Machine Learning Basics

k-nearest neighbor (k-NN) – Majority voting의 문제점

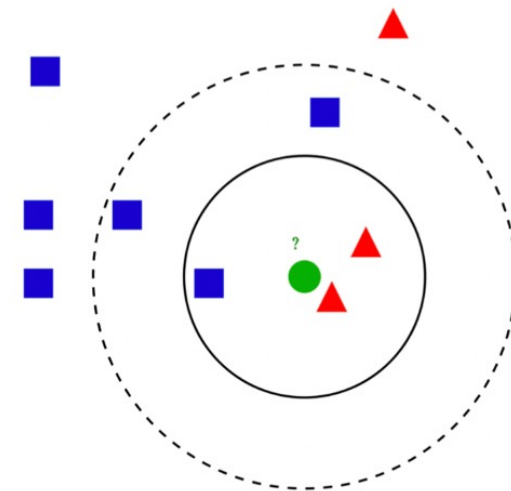
- kNN은 영역안에 많은 것이 카테고리로 선택되는 시스템을 가짐
- 기본적으로 어떤 카테고리가 타 카테고리보다 많이 있다면 분류에 상당한 영향을 끼침
- 예를 들어, 남자 10명, 여자가 90명이라면 대부분 여자로 예측할 가능성이 큼

Distance-Weighted Voting

kNN은, 분류와 회귀 모두에서, 주변 값들의 기여도에 가중(weight)을 부여할 수 있음.

→ 더 가까운 주변일수록 더 큰 가중을 부여할 수도 있음.

예) 각 주변점에 대해 거리의 역수($1/d$)를 가중치로 사용



Machine Learning Basics

k-nearest neighbor (k-NN)

장점

알고리즘이 쉽고 직관적

수치기반 데이터 분류 작업에서 어느 정도 noise가 있어도 성능이 괜찮음

단점

학습 데이터 양(k값이 클수록)이 많으면 속도가 느려진다.

차원(벡터)의 크기가 크면 계산량이 많아진다.

거리기반 알고리즘이므로 표준화가 필요하다.

효율성에 있어서 매번 모든 데이터를 활용해야 한다.

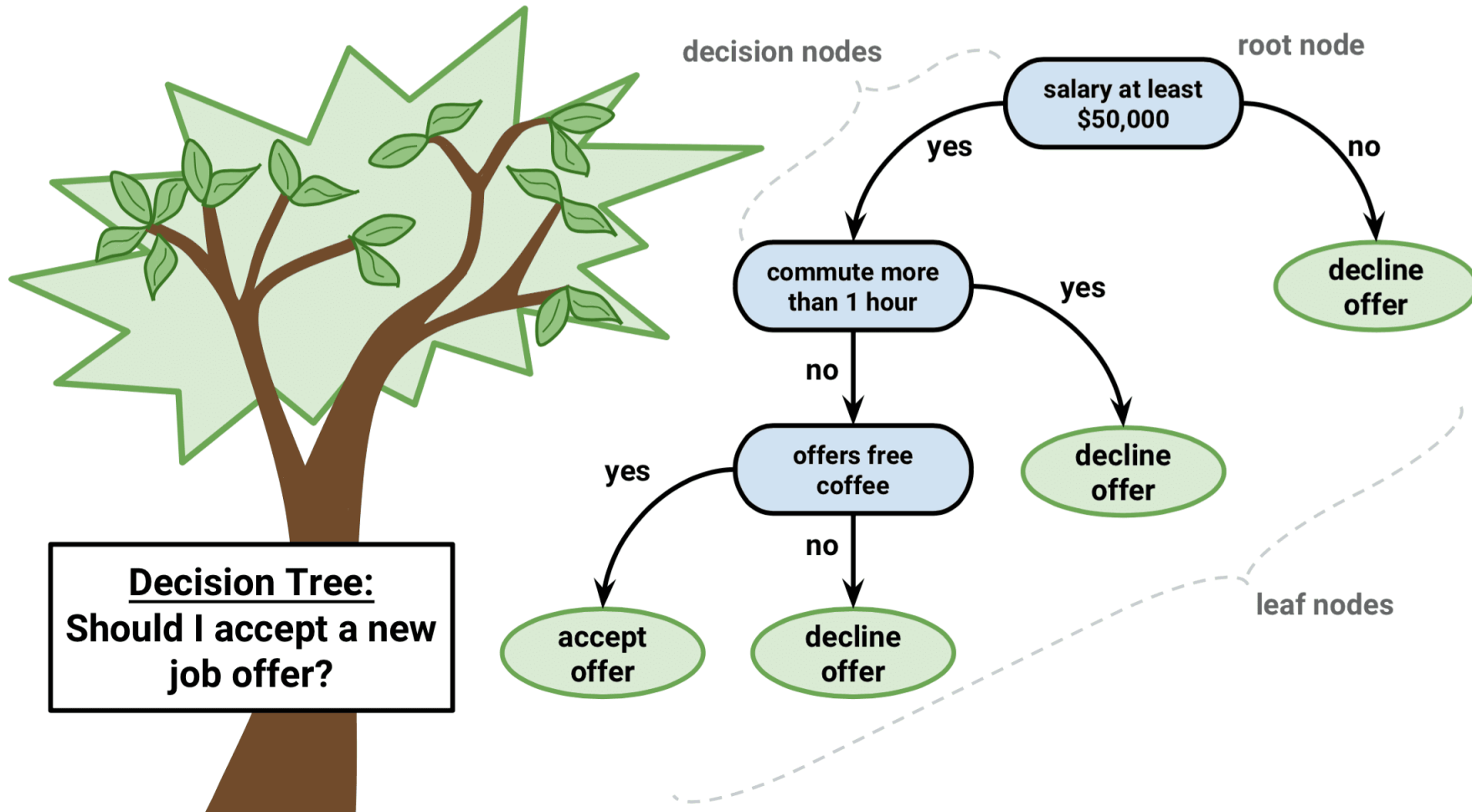
Machine Learning Basics

의사결정나무, Decision Tree

- 머신러닝 알고리즘 중 지도학습.
- 스무고개 하듯이 예/아니오 질문을 반복하여 학습.
- 모양이 나무를 닮아서 의사결정 나무라 부름.
- 의사결정나무는 의사결정 규칙을 나무 구조로 나타내어 전체 자료를 몇 개의 소집단으로 분류하여 예측하는 분석 방법.
- 분류(classification)와 회귀(regression) 모두 사용 가능.
- 데이터 분류와 예측에서 강력하고 인기있는 도구
 - 인간이 쉽게 이해할 수 있는 언어로 표현할 수 있는 규칙을 기반으로 판단
 - 규칙을 통해 모델을 이해할 수 있어 설명력이 있는 모델

Machine Learning Basics

의사결정나무, Decision Tree



출처: <https://dataaspirant.com>

Machine Learning Basics

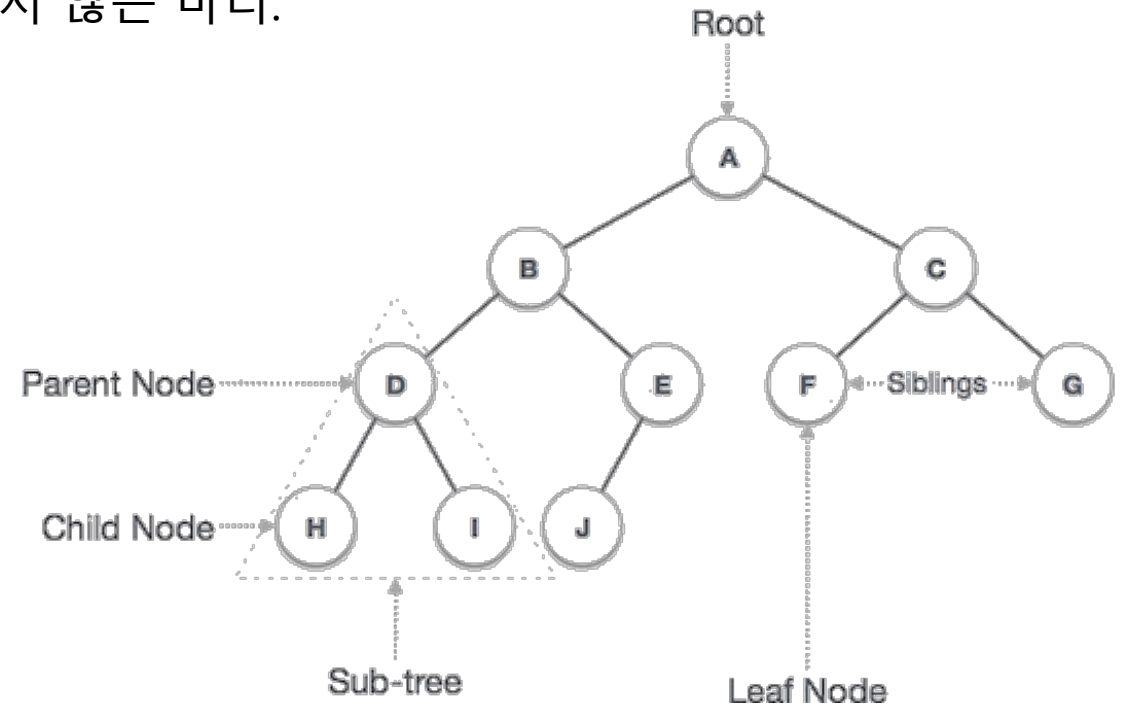
의사결정나무, Decision Tree

- 특정 기준(질문)에 따라 데이터를 구분하는 모델.
- 상위 노드로부터 하위 노드로 나무 구조를 형성하는 매 단계마다 분류변수와 분류기준값의 선택이 중요.
각 노드마다 질문을 던지고 그 응답에 따라 가지를 쳐서 데이터를 분리.
- 데이터로부터 트리구조의 일반화된 지식을 추출
- 데이터가 얼마나 잘 분리되었는지를 평가하기 위한 기준 : 불순도(impurity)
노드에 여러 분류가 섞여있을수록 불순도는 높다.
노드에 하나의 분류만 존재할 때 가장 낮은 불순도를 갖는다.
노드 분리 후 각 노드의 불순도가 낮아질수록 트리분류가 잘 된 것.

Machine Learning Basics

의사결정나무, Decision Tree

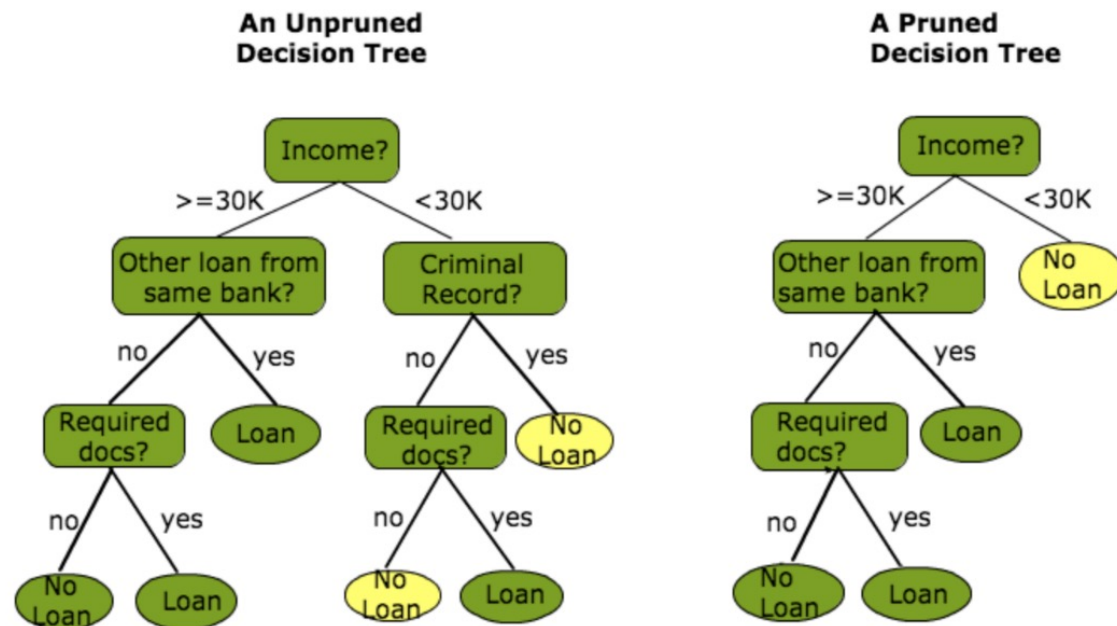
- 뿌리 마디(root node) : 제일 상위의 마디. 분류 대상이 되는 모든 자료집단을 포함.
- 부모 마디(parent node) : 상위 마디가 하위 마디로 분기 될 때의 상위 마디.
- 자식 마디(child node) : 상위 마디가 하위 마디로 분기 될 때의 하위 마디.
- 최종 마디(terminal node, leaf node) : 더이상 분기되지 않는 마디.



Machine Learning Basics

의사결정나무, Decision Tree

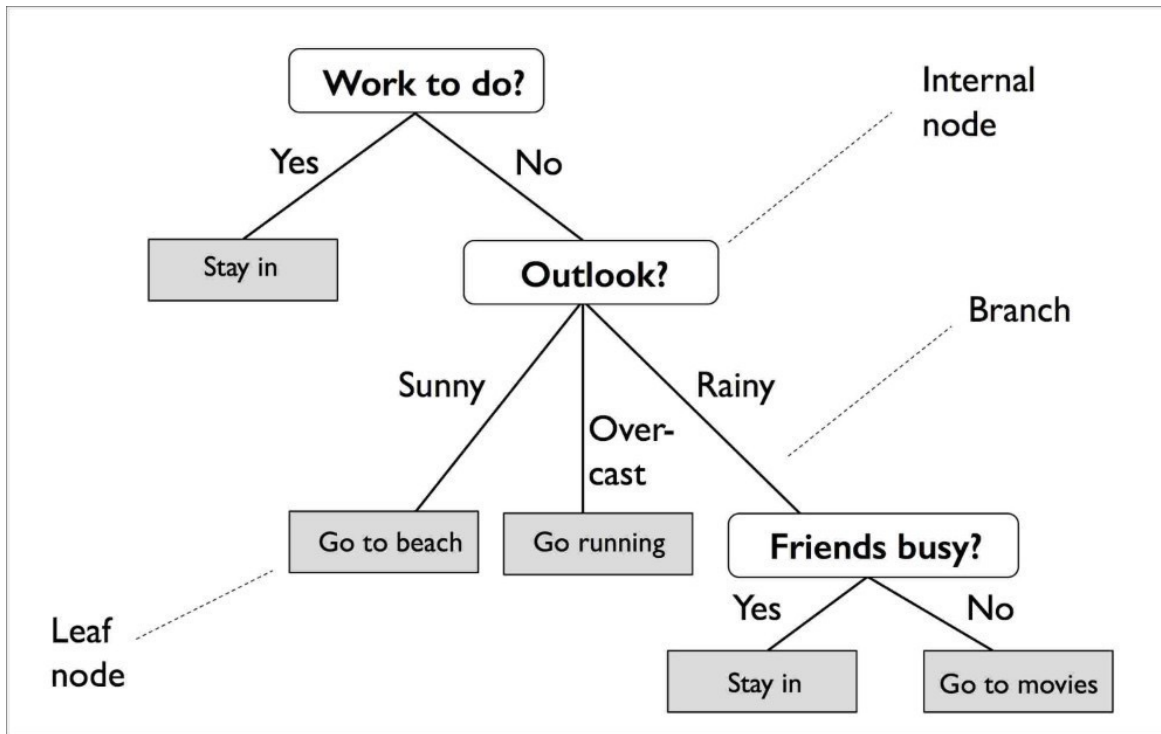
- 상위 노드로부터 하위노드로 트리구조를 형성하는 때 단계마다 분류 변수와 기준값의 선택이 중요함.
- 가지분할(split) : 나무의 가지를 생성하는 과정.
- 가지치기(pruning) : 생성된 가지를 잘라내어 나무의 형태를 단순화하는 과정.
- Decision node를 통해 규칙이 되는 조건을 알 수 있음



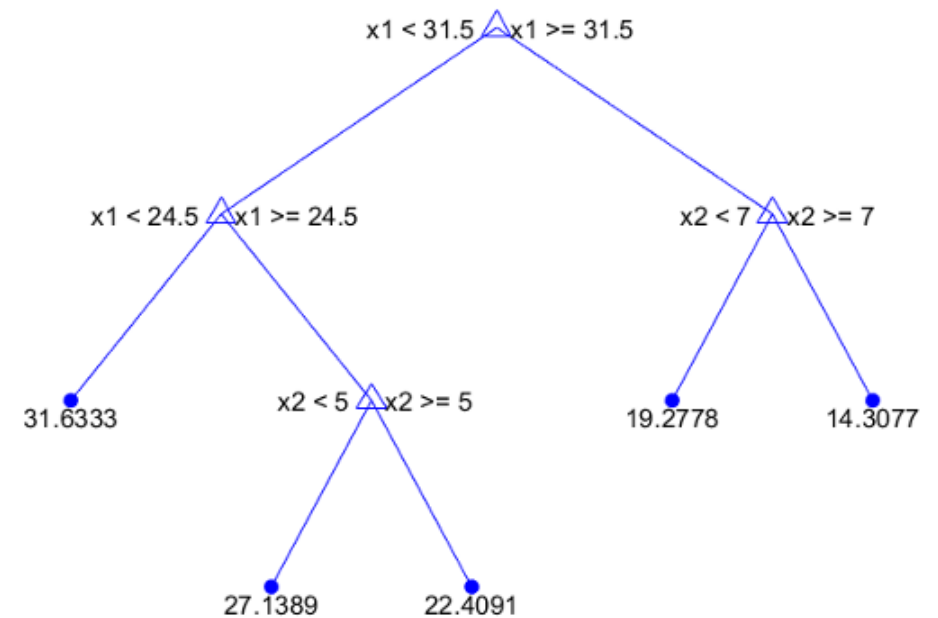
Machine Learning Basics

의사결정나무, Decision Tree

- 범주형: 분류나무(Classification Tree): 목표 변수가 유한한 수의 값을 가지면 분류목적의 분류 나무
- 연속형: 회귀나무(Regression Tree): 목표 변수가 실수값을 가지면 수치 예측 목적의 회귀 나무



출처: <https://medium.com/analytics-vidhya/7-types-of-multi-classification-using-python-ba524827f250>

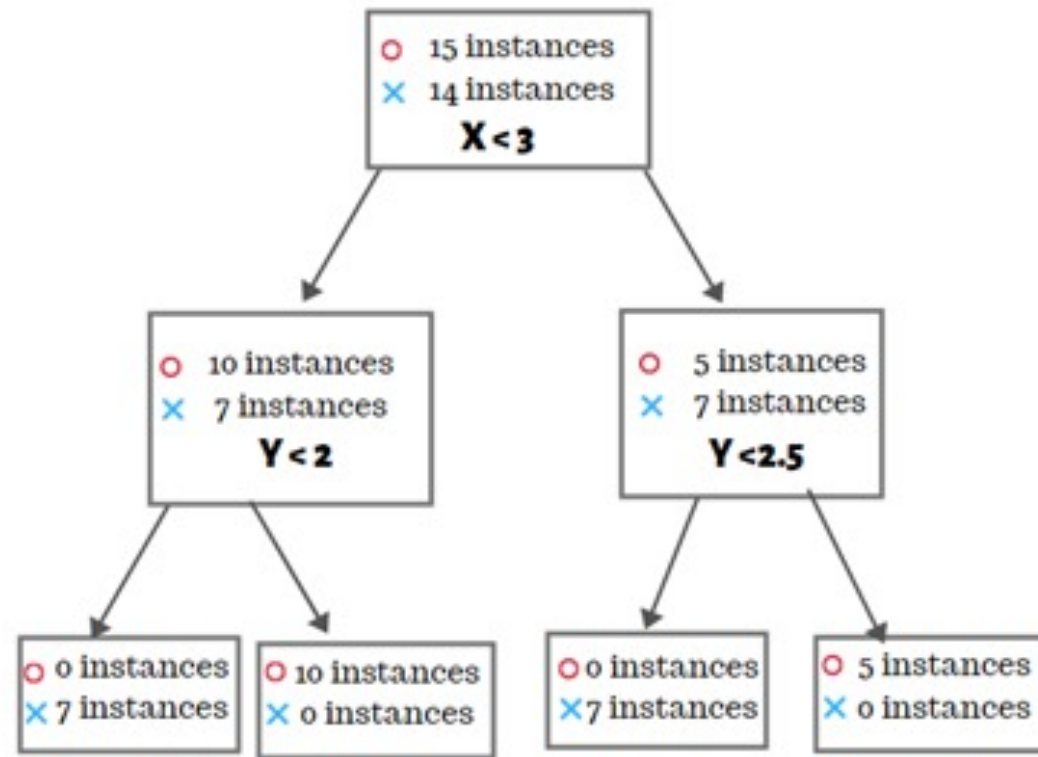
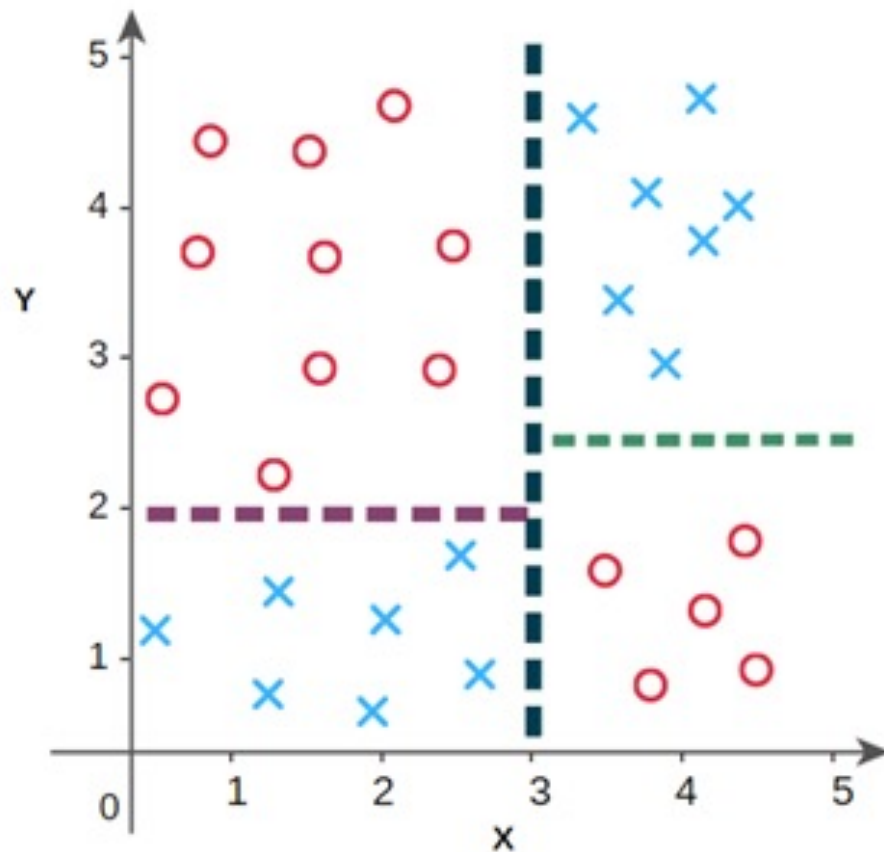


출처: <https://kr.mathworks.com/help/stats/train-regression-trees-using-regression-learner-app.html>

Machine Learning Basics

의사결정나무, Decision Tree

- 의사결정나무를 통한 비선형 데이터 분류



Machine Learning Basics

의사결정나무, Decision Tree

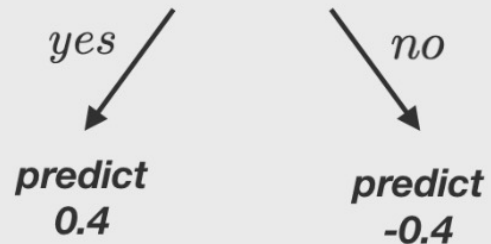
- 의사결정나무를 통한 회귀

depth 0 tree

predict 0.0
(mean of y values)

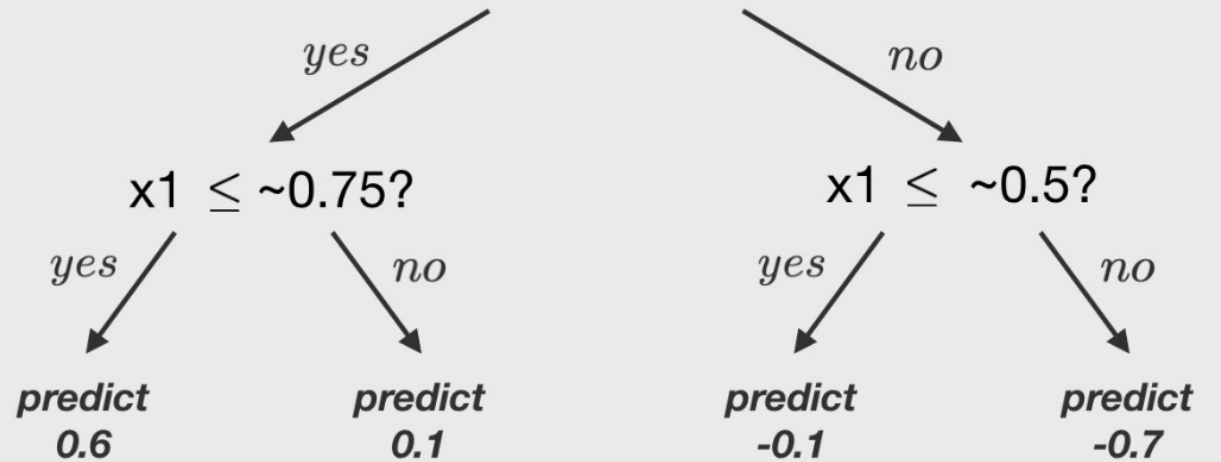
depth 1 tree

$x_2 \leq \sim 0.8?$



depth 2 tree

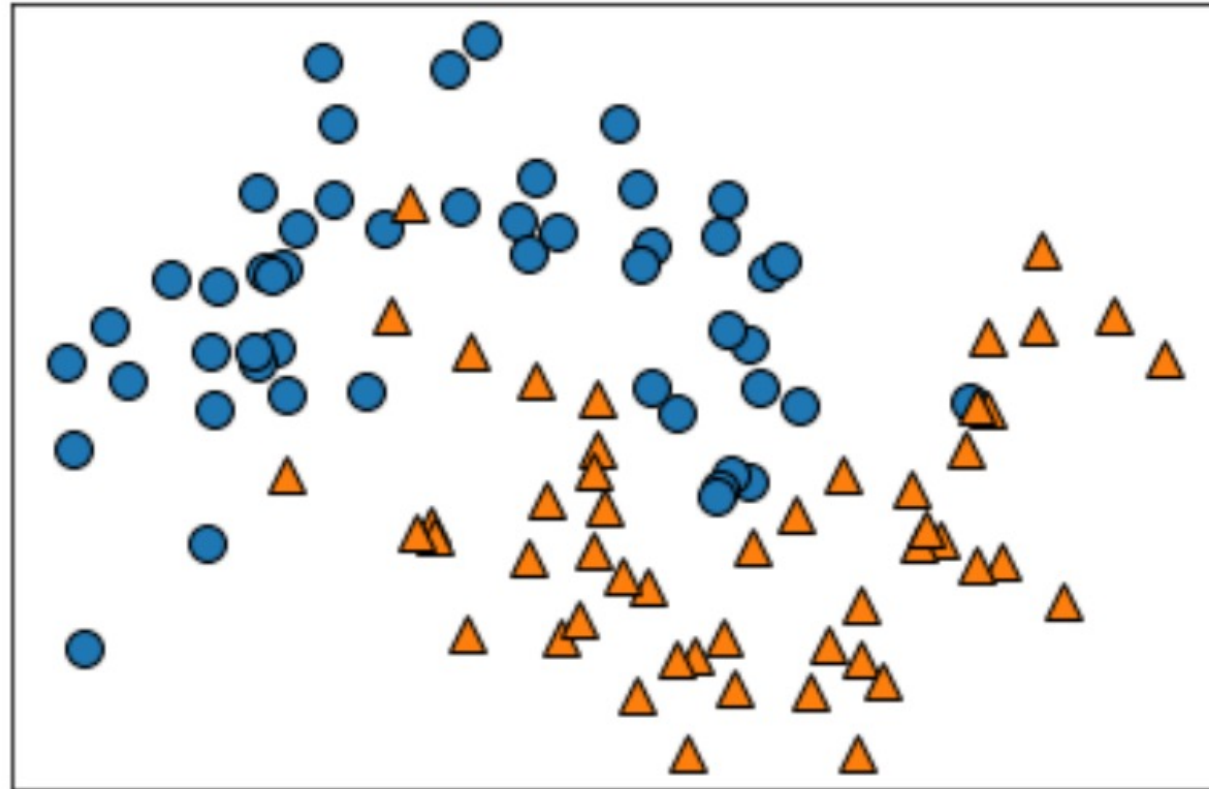
$x_2 \leq \sim 0.8?$



Machine Learning Basics

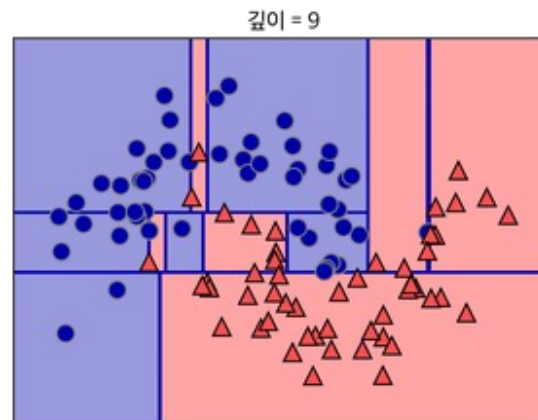
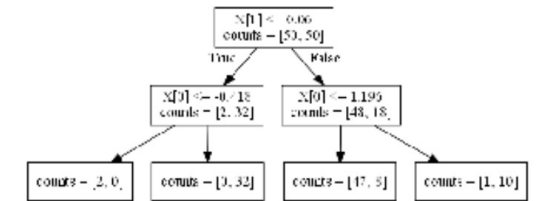
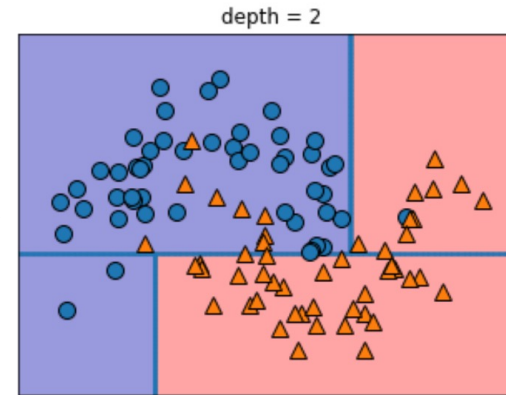
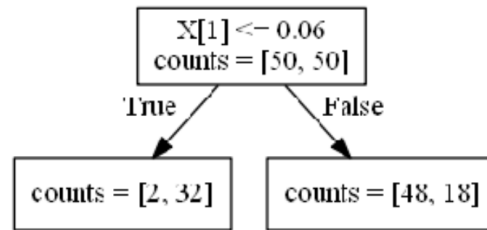
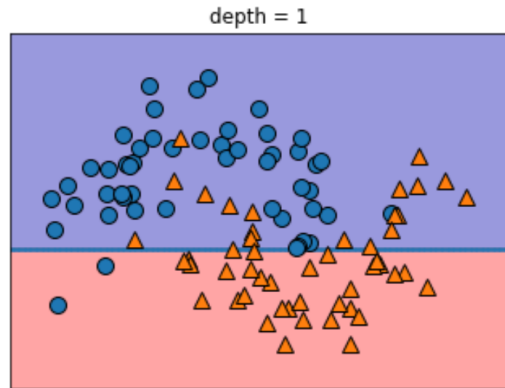
의사결정나무, Decision Tree

- 아래의 비선형 관계를 갖는 2차원 데이터는 logistic regression이나 SVM으로 분류하기가 어려워 보이지만, decision tree는 쉽게 분류 가능



Machine Learning Basics

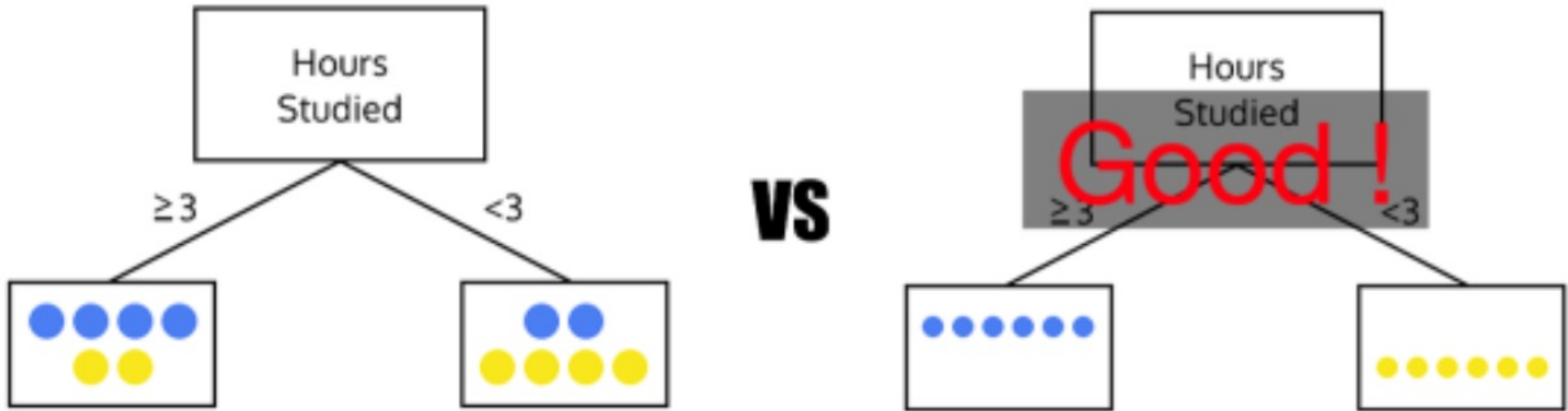
의사결정나무, Decision Tree

출처: <https://bkshin.tistory.com/entry/%EB%A8%B8%EC%8B%A0%EB%9F%AC%EB%8B%9D-4-%EA%B2%B0%EC%A0%95-%ED%8A%B8%EB%A6%ACDecision-Tree?category=1057680>

Machine Learning Basics

의사결정나무, Decision Tree

- 어떤 변수와 기준 값으로 데이터를 나눠야 할까?

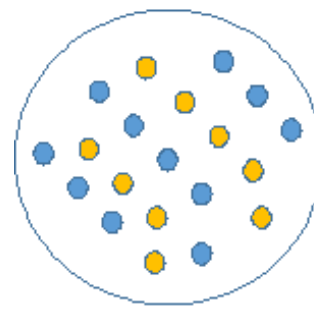


Machine Learning Basics

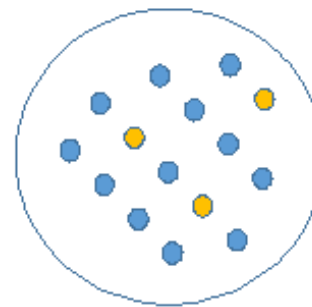
의사결정나무, Decision Tree

- 데이터가 얼마나 잘 분리되었는지를 평가하기 위한 기준 : 불순도(impurity)
 - 노드에 여러 분류가 섞여있을수록 불순도는 높다.
 - 노드에 하나의 분류만 존재할 때 가장 낮은 불순도를 갖는다.
 - 노드 분리 후 각 노드의 불순도가 낮아질수록 트리분류가 잘 된 것.
- 상위 노드에서의 분류변수와 분류 기준값은 이 기준에 의해 분기되는 하위노드에서 노드(집단) 내에서의 동질성이, 노드(집단)간에는 이질성이 가장 커지도록 선택됨.
- 의사결정 나무를 만들 때에는, 각 노드들의 복잡성, 즉 불순도(impurity)가 가장 낮은 방향으로 tree가 만들어짐
- 나무 모형의 크기는 과대적합(또는 과소적합) 되지 않도록 합리적 기준에 의해 적당히 조절되어야 함.

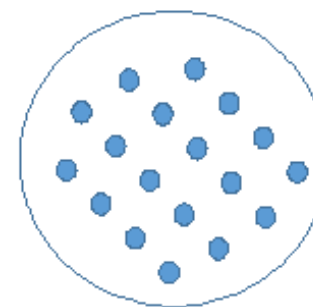
불순도(A) > 불순도(B) > 불순도(C)



A



B



C

Machine Learning Basics

의사결정나무, Decision Tree – 불순도 종류

불순도 종류	분류나무(Classification tree)	회귀나무(Regression tree)
분류변수와 분류기준값 선택방법	카이제곱통계량의 p값 <u>지니계수</u> 엔트로피 지수	F 통계량의 p값 분산의 감소량 목표변수의 평균과 표준편차 등 불순도

Machine Learning Basics

의사결정나무, Decision Tree – 불순도, Gini-Index

- 해당 범주 안에 서로 다른 데이터가 얼마나 섞여 있는지를 나타냄.
- 의사결정 나무 모델의 노드 분할 기준이 됨.
- 0과 0.5 사이 값을 범위로 가짐.

지니계수가 0에 가까울수록 잘 분류된 것 (좋은 질문, 분리 기준)

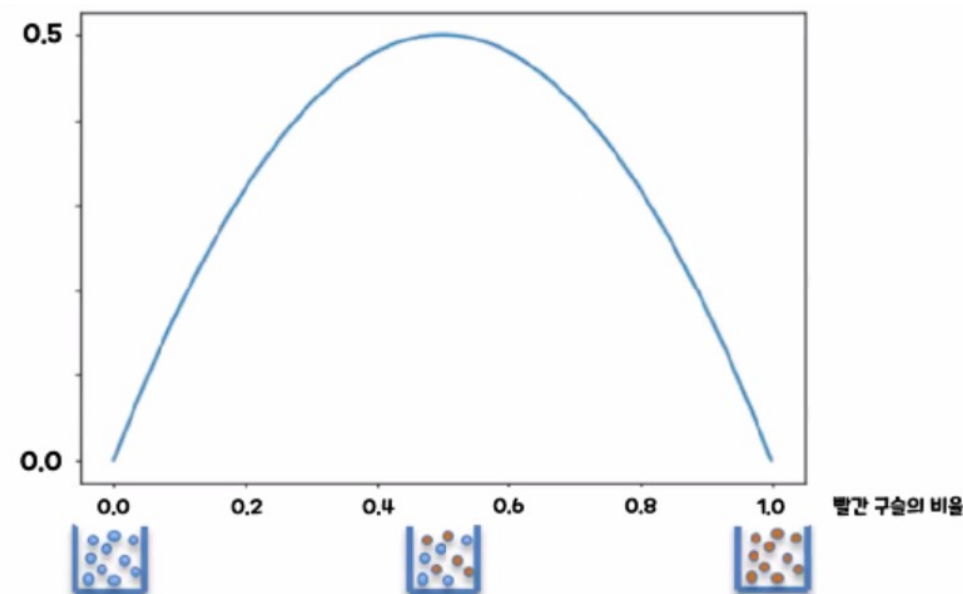
0.5라면 데이터가 5:5 비율로 섞여서 분류된 것 (좋지 않은 질문, 분리 기준)

- 분할시 GINI index가 낮아지는 방향을 선택함.

$$GI_{node} = 1 - \sum_{k=1}^m p_{ik}^2$$

$$GI_{split} = \sum_{i=1}^d (R_i (1 - \sum_{k=1}^m p_{ik}^2))$$

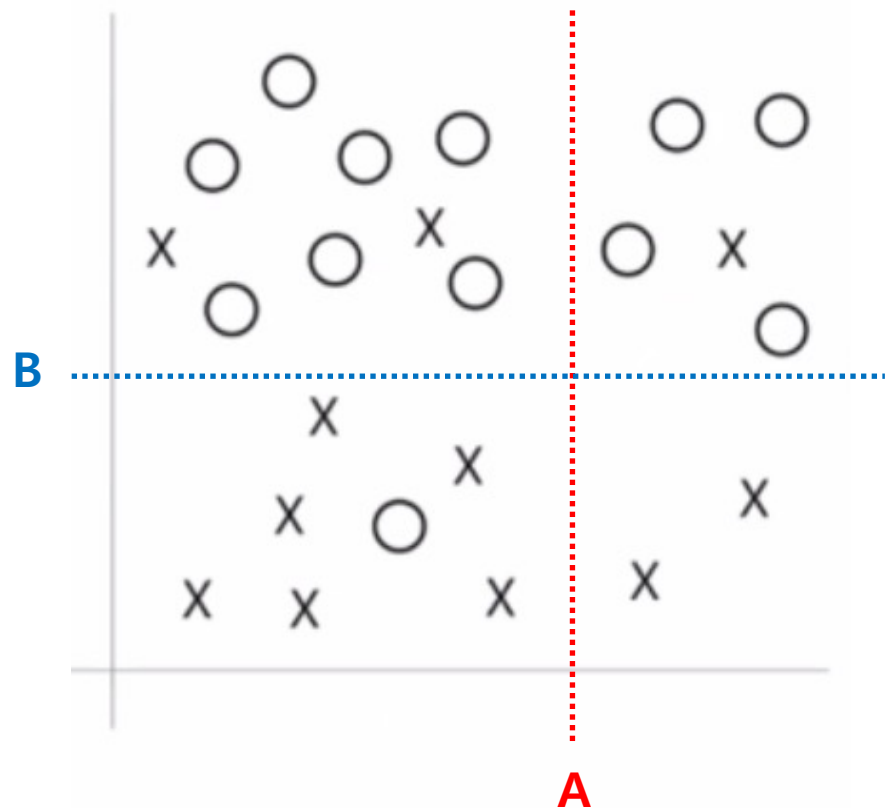
(R_i 는 분할 전 데이터 가운데 분할 후 i 영역에 속하는 데이터의 비율)



Machine Learning Basics

의사결정나무, Decision Tree – 불순도, Gini-Index

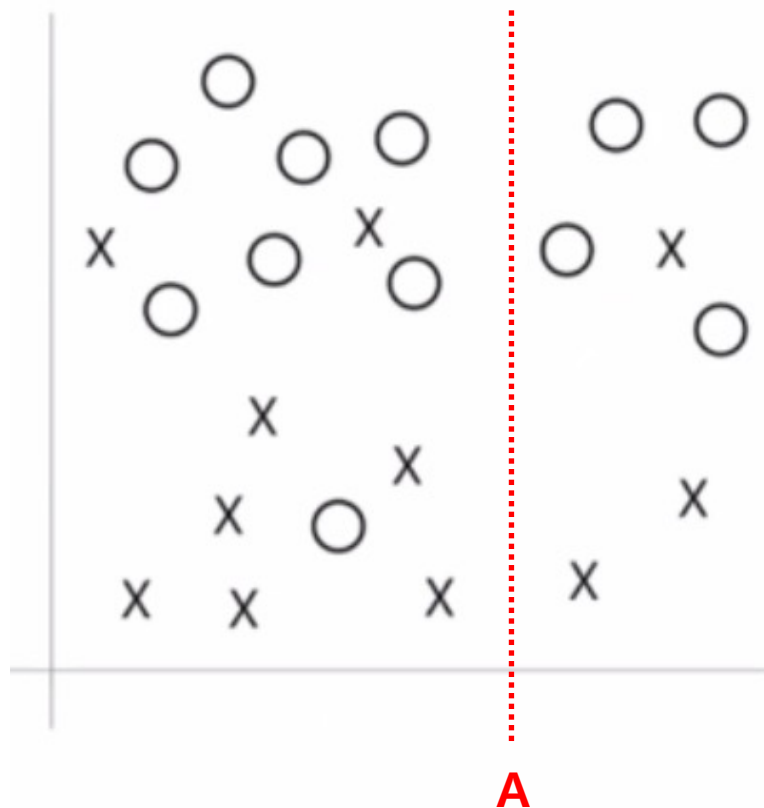
- 분리 기준 A와 B가 있을 때 Gini-Index 비교



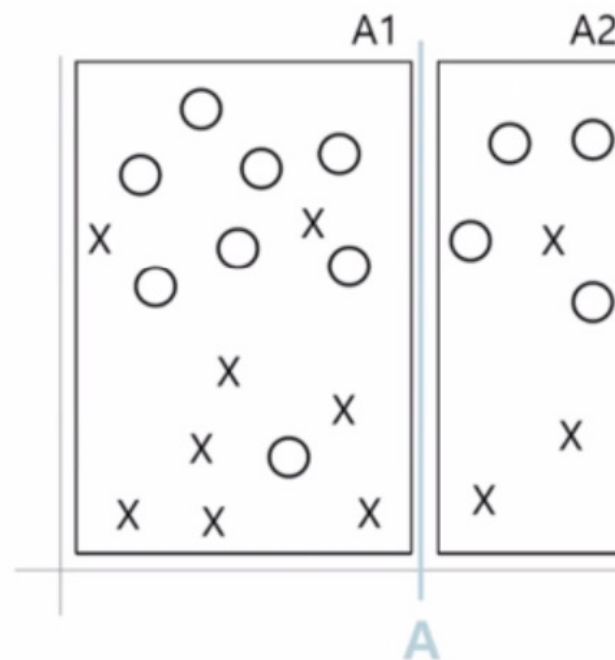
Machine Learning Basics

의사결정나무, Decision Tree – 불순도, Gini-Index

- 분리 기준 A와 B가 있을 때 Gini-Index 비교



· A를 기준으로 분할



$$GI_{node} = 1 - \sum_{k=1}^m p_{ik}^2$$

$$GI_{split} = \sum_{i=1}^d (R_i (1 - \sum_{k=1}^m p_{ik}^2))$$

A1에서 O의 비율 A1에서 X의 비율

$$G_{A1} = 1 - \left(\frac{8}{16}\right)^2 - \left(\frac{8}{16}\right)^2 = 0.5$$
$$G_{A2} = 1 - \left(\frac{4}{7}\right)^2 - \left(\frac{3}{7}\right)^2 = 0.49$$
$$G_A = \left(\frac{16}{23}\right) * 0.5 + \left(\frac{7}{23}\right) * 0.49 = 0.497$$

R_{A1} R_{A2}

Machine Learning Basics

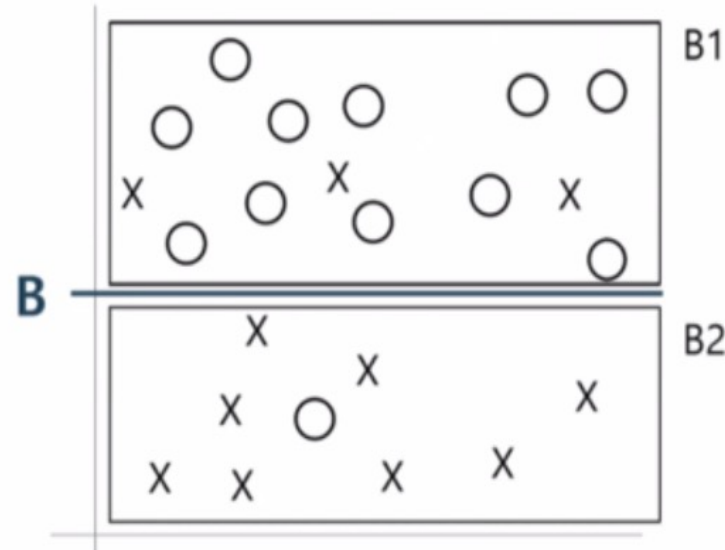
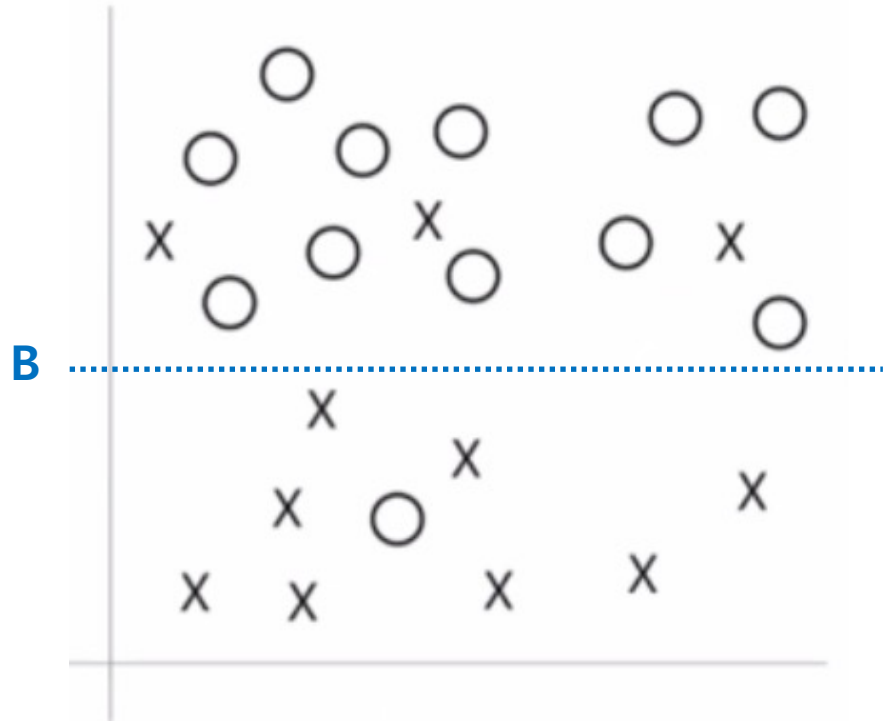
의사결정나무, Decision Tree – 불순도, Gini-Index

- 분리 기준 A와 B가 있을 때 Gini-Index 비교

$$GI_{node} = 1 - \sum_{k=1}^m p_{ik}^2$$

$$GI_{split} = \sum_{i=1}^d (R_i (1 - \sum_{k=1}^m p_{ik}^2))$$

B를 기준으로 분할



$$G_{B1} = 1 - \left(\frac{11}{14}\right)^2 - \left(\frac{3}{14}\right)^2 = 0.34$$

$$G_{B2} = 1 - \left(\frac{1}{9}\right)^2 - \left(\frac{8}{9}\right)^2 = 0.2$$

$$G_B = \left(\frac{14}{23}\right) * 0.34 + \left(\frac{9}{23}\right) * 0.2 = 0.28$$

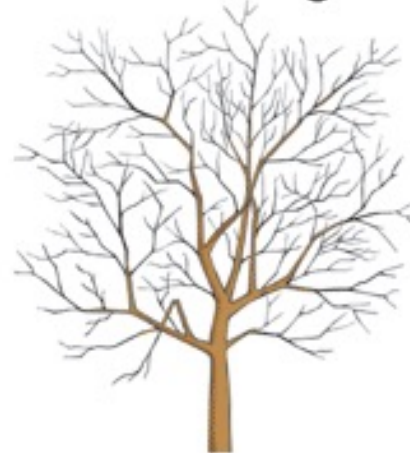
→ B기준으로 split!

Machine Learning Basics

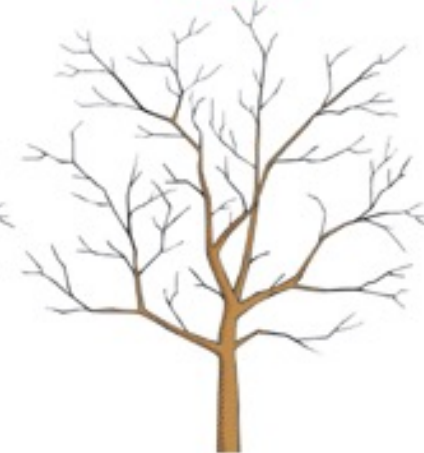
의사결정나무, Decision Tree – 가지치기(pruning)

- 의사결정 나무의 분기 수(깊이:depth)가 증가할 때 처음에는 새로운 데이터에 대한 오분류율이 감소하나 일정 수준 이상이 되면 오분류율이 오히려 증가하는 현상이 발생함.
- 이러한 문제를 해결하기 위해서는 검증데이터에 대한 오분류율이 증가하는 시점에서 적절히 가지치기를 수행해줘야 함.
- 가지치기는 의사결정나무에서 쓸모없는 규칙을 가지고 있는 가지들을 제거하여 모델을 다듬어 주는 것 (overfitting 방지)

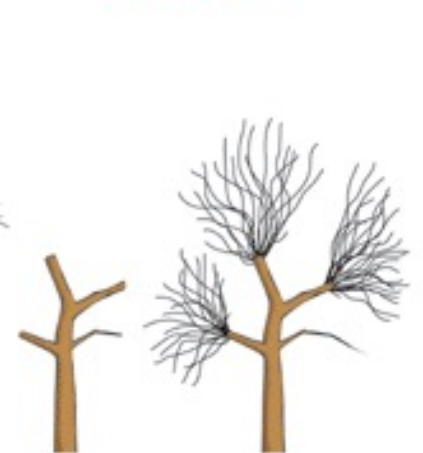
A Look at Pruning



GOOD

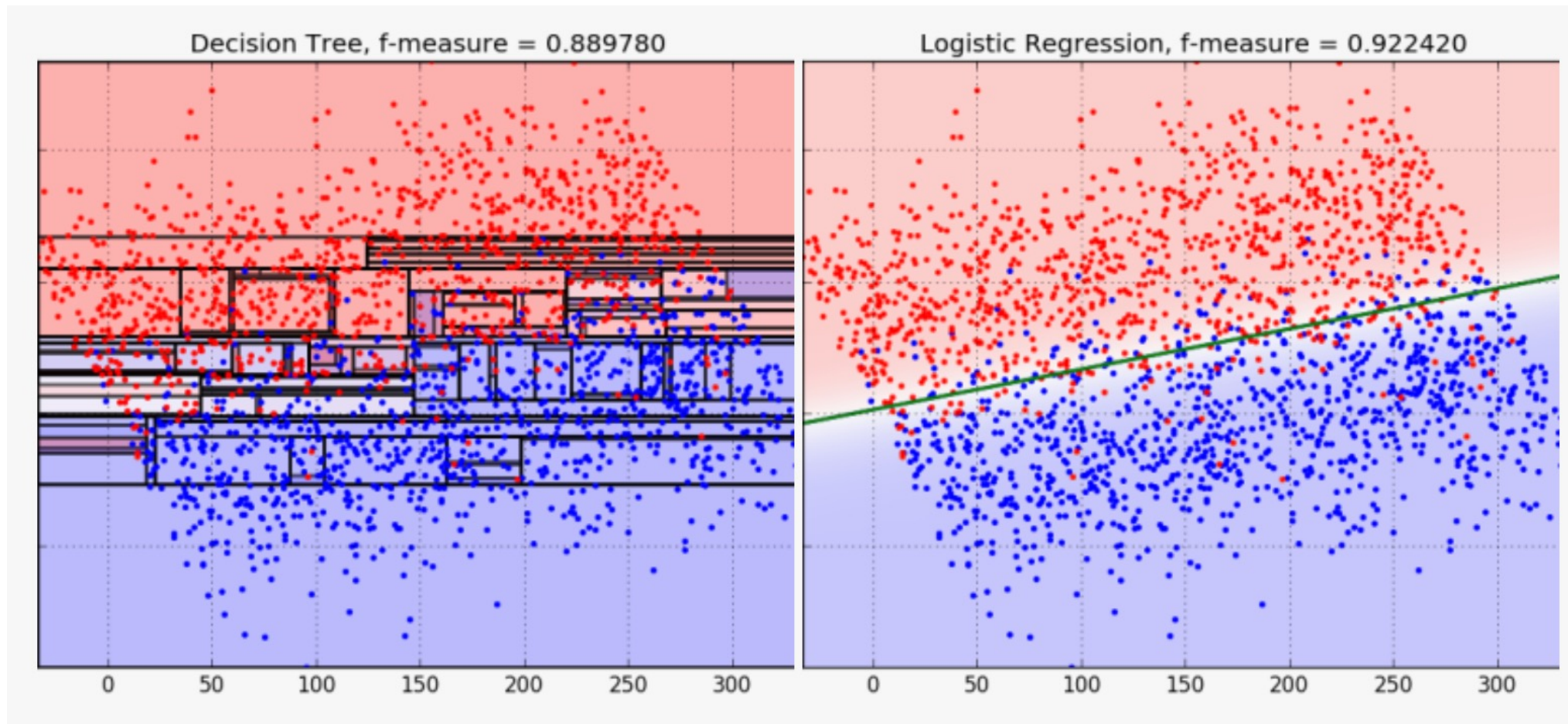


NOT GOOD



Machine Learning Basics

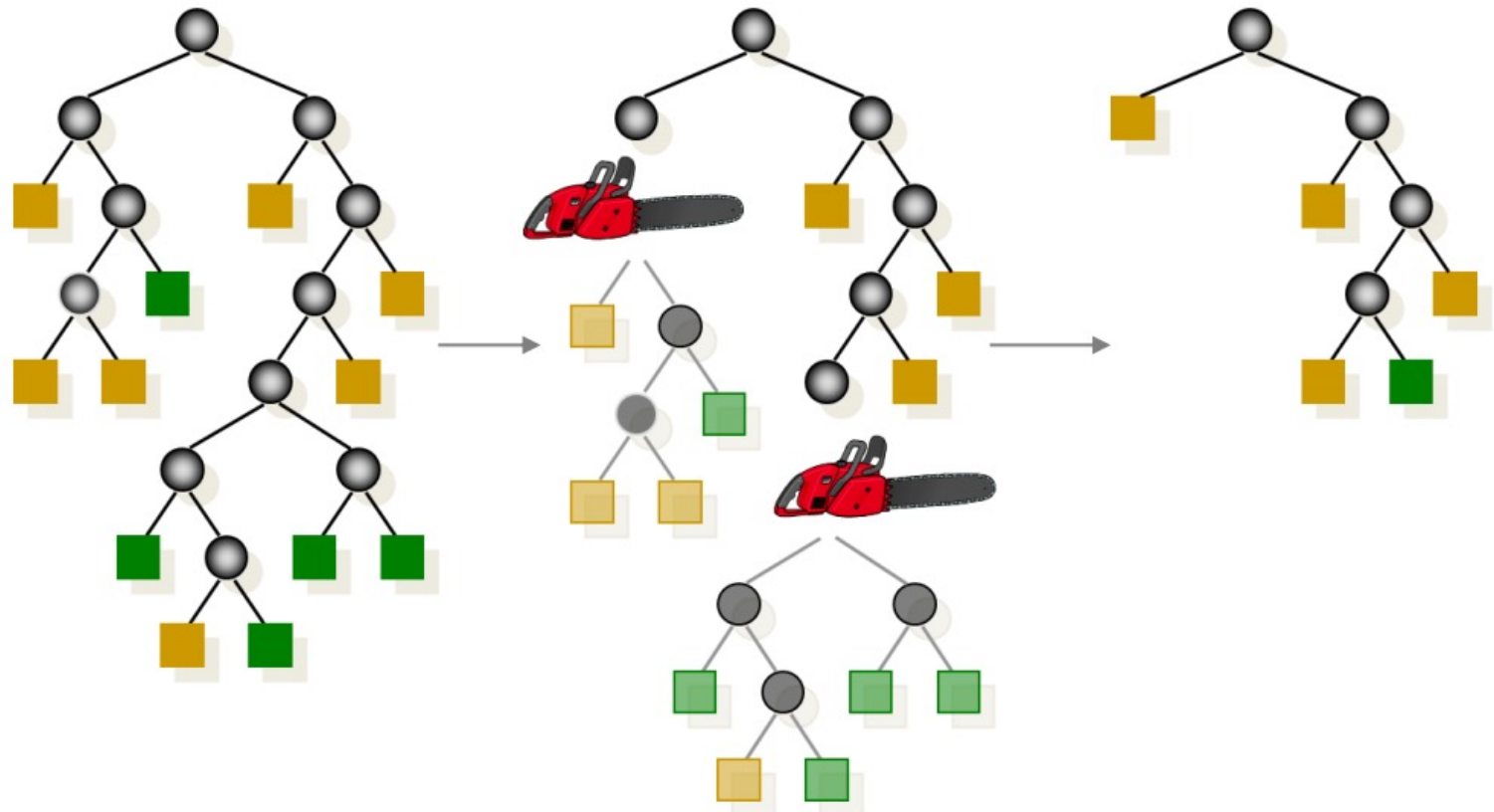
의사결정나무, Decision Tree – 가지치기(pruning)



Machine Learning Basics

의사결정나무, Decision Tree – 가지치기(pruning)

- 마치 나뭇가지를 잘라내는 것과 같다는 의미에서 가지치기(Pruning)라 부름.
- 가지치기는 데이터를 버리는 개념이 아니고 분기를 합치는(merge) 개념으로 이해해야 함



출처: <https://ratsgo.github.io/machine%20learning/2017/03/26/tree/>

Machine Learning Basics

의사결정나무, Decision Tree

의사결정나무 알고리즘의 장점

- 분류와 회귀 문제 모두에 사용할 수 있음
- 데이터를 조건에 따라 나누는 방식이라 결과를 해석하기 쉬움
- 선형성, 정규성, 다중공선성 같은 가정이 거의 필요 없음
- 수치형 변수와 범주형 변수를 모두 사용할 수 있음
- 변수 간 비선형 관계나 상호작용 효과를 잘 반영

의사결정나무 알고리즘의 단점

- 데이터가 조금만 바뀌어도 트리 구조가 크게 달라질 수 있음
- 트리가 너무 깊어지면 과적합이 발생하기 쉬움
- 연속형 변수를 구간으로 나누기 때문에 경계 근처에서 예측이 불안정할 수 있음
- 회귀 문제에서는 예측값이 계단형으로 나타나 부드러운 관계를 표현하기 어려움
- 트리가 커지면 오히려 해석이 어려워짐

Machine Learning Basics

앙상블 모델(Ensemble Model)

- 여러 개의 학습 모델을 결합하여 하나의 모델보다 더 안정적이고 정확한 예측을 수행하는 방법
- 개별 모델은 각각 다른 관점에서 데이터를 학습하며, 이들의 예측 결과를 투표하거나 평균 내어 최종 결과를 결정
- 대표적인 앙상블 방법에는 여러 모델을 독립적으로 학습하는 배깅(Bagging), 순차적으로 오차를 보완하는 부스팅(Boosting)이 있음

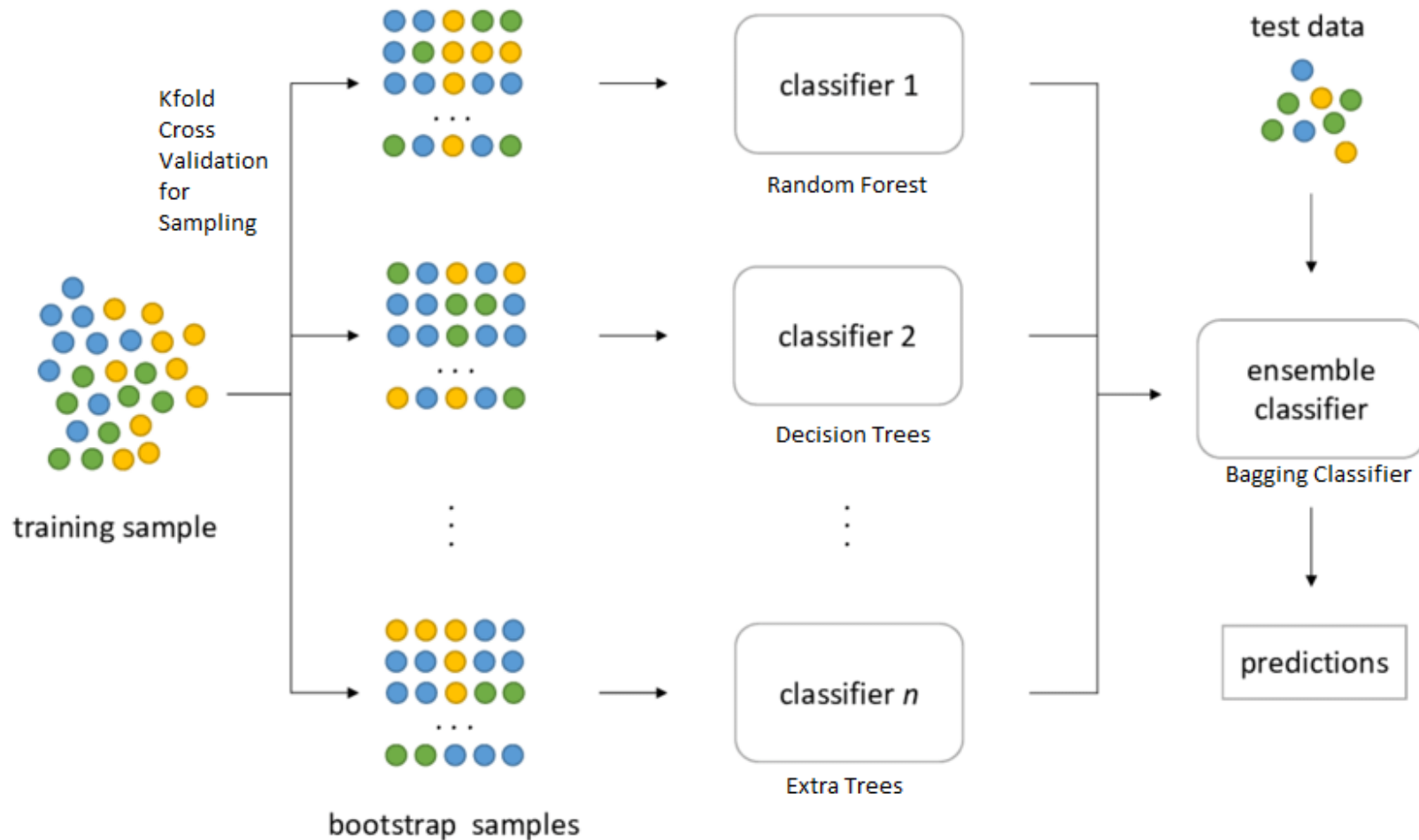
Machine Learning Basics

앙상블 모델(Ensemble Model) – Bagging

- Bootstrap Aggregating
- 샘플을 여러 번 뽑아 각 모델을 학습시켜 결과를 집계(Aggregating) 하는 방법
 - 먼저 대상 데이터로부터 복원 랜덤 샘플링(Bootstrap)을 함. 이렇게 추출한 데이터가 일종의 표본 집단이 됨.
 - 이제 여기에 동일한 모델을 학습시킴.
 - 그리고 학습된 모델의 예측변수들을 집계(Aggregating)하여 그 결과로 모델을 생성.
- Bagging은 각 샘플에서 나타난 결과를 일종의 중간값으로 맞추어 주기 때문에, 모델 성능의 variance를 줄여 Overfitting을 피할 수 있음
- 일반적으로 Categorical Data인 경우, 투표 방식으로 집계하며 Continuous Data인 경우, 평균으로 집계함.
- 대표적인 Bagging 알고리즘으로 Random Forest 모델이 있음.

Machine Learning Basics

앙상블 모델(Ensemble Model) – Bagging



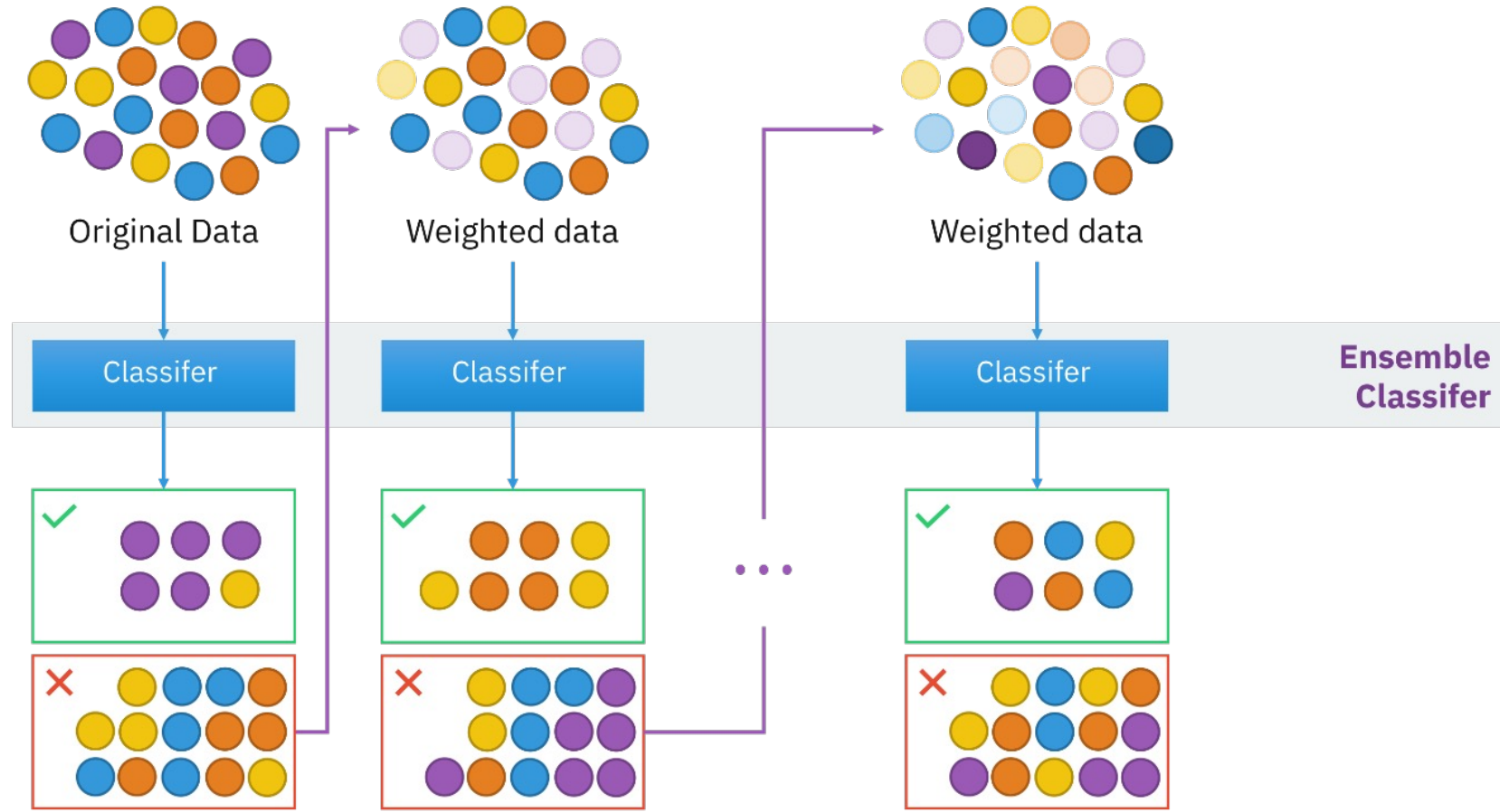
Machine Learning Basics

앙상블 모델(Ensemble Model) – Boosting

- Boosting은 **가중치를 활용하여** 약 분류기를 강 분류기로 만드는 방법임
- Bagging의 경우 각각의 독립적인 모델들을 결합하여 최종 결과 값을 예측하는 것인데, Boosting 은 모델간 연결고리가 생겨 팀워크가 이뤄진다고 보면 됨.
- 분류 오답에 대해 높은 가중치를 부여하고, 정답에 대해 낮은 가중치를 부여하기 때문에 오답에 더욱 집중할 수 있게 되어, 정확도가 높게 나타남. 하지만, 그만큼 outlier에 취약하기도 함.
- AdaBoost, XGBoost, GradientBoost 등 다양한 모델이 있음.
- 그 중에서도 XGBoost 모델은 강력한 성능을 보여줌. (최근 대부분의 Kaggle 대회 우승 알고리즘)

Machine Learning Basics

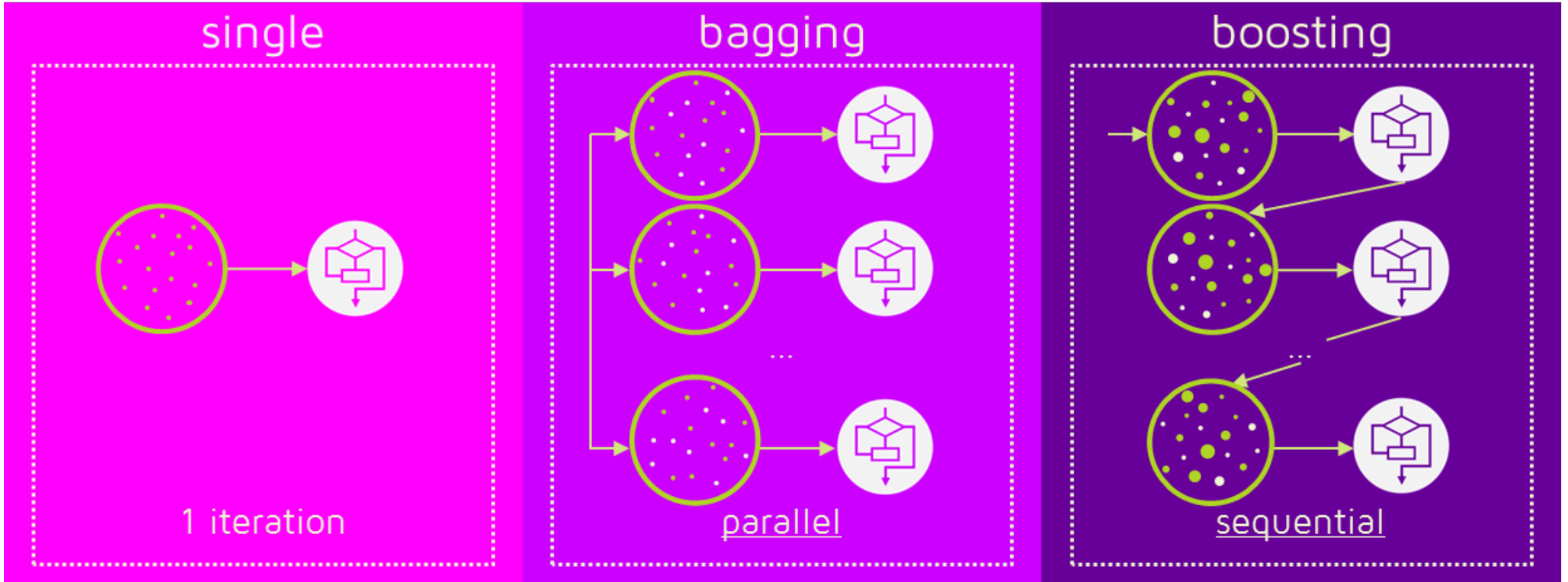
앙상블 모델(Ensemble Model) – Boosting



출처: [https://en.wikipedia.org/wiki/Boosting_\(machine_learning\)#/media/File:Ensemble_Boosting.svg](https://en.wikipedia.org/wiki/Boosting_(machine_learning)#/media/File:Ensemble_Boosting.svg)

Machine Learning Basics

앙상블 모델(Ensemble Model) – Boosting vs Bagging



Machine Learning Basics

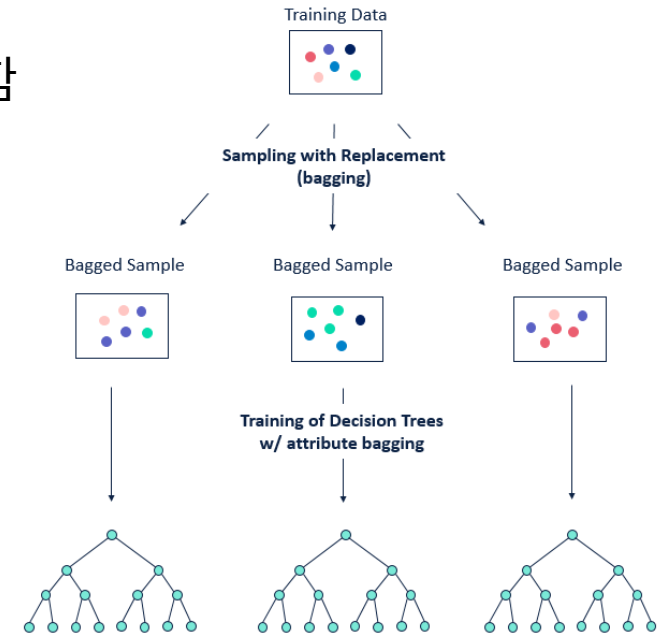
앙상블 모델(Ensemble Model) – Boosting vs Boosting

- Boosting도 Bagging과 동일하게 복원 랜덤 샘플링을 하지만, 가중치를 부여한다는 차이점이 있음.
- Bagging이 병렬로 학습하는 반면, Boosting은 순차적으로 학습시킴. 학습이 끝나면 나온 결과에 따라 가중치가 재분배됨.
- 개별 의사결정 나무의 성능이 낮은 것이 문제라면 boosting 방법을 이용.
- 개별 의사결정 나무의 과대적합이 문제라면 bagging 방법을 이용.

Machine Learning Basics

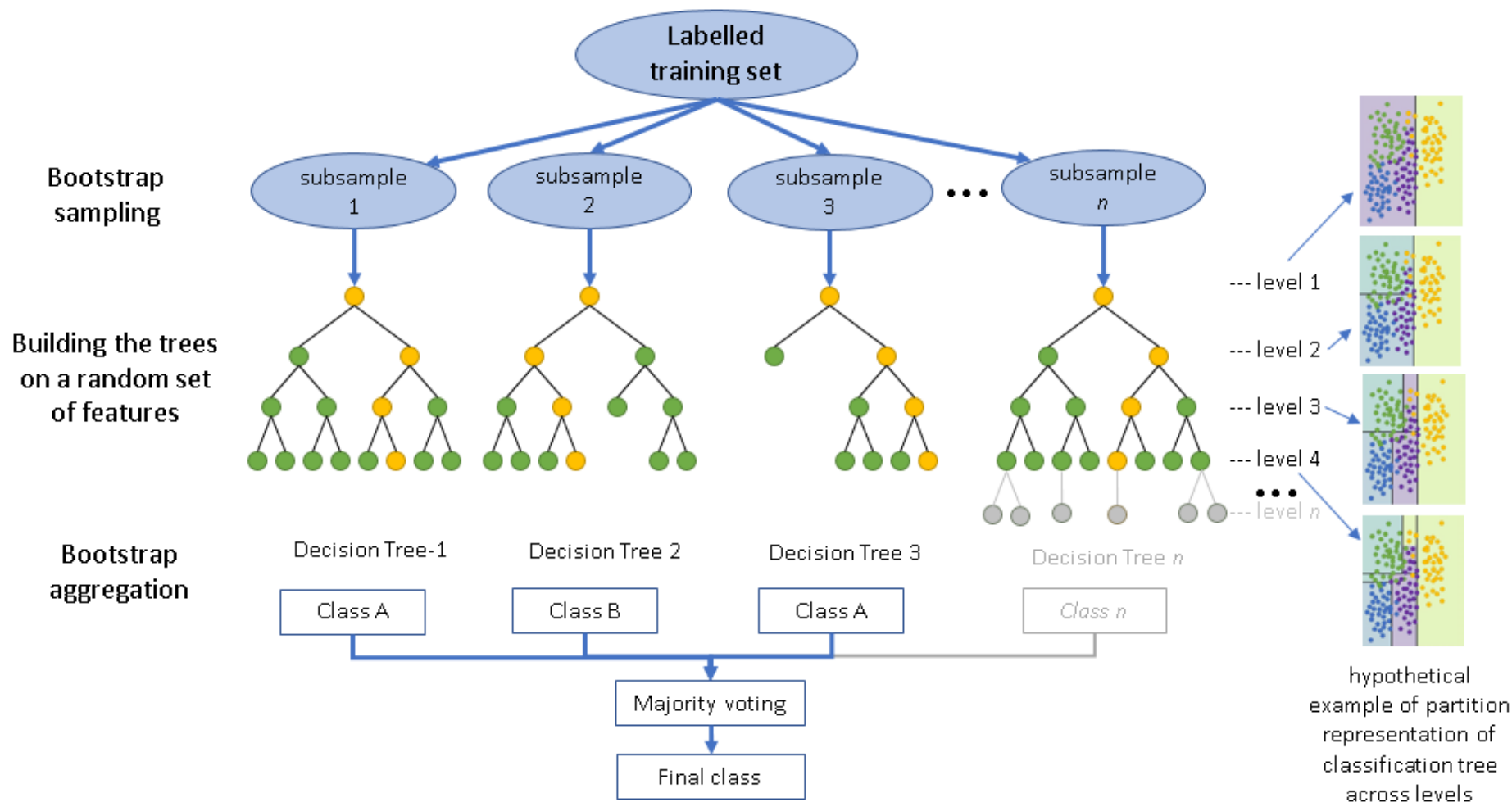
앙상블 모델(Ensemble Model) – Bagging → Random Forest

- 앙상블 머신러닝 모델 (Ensemble machine learning model)
 - 주어진 데이터로부터 여러 개의(weak) 모델들을 학습
 - 예측시 여러 모델들의 예측 결과들을 종합해 사용하여 정확도를 높이는 기법
- 의사결정 나무(decision tree)와 배깅(Bagging)을 결합한 알고리즘
- 여러 개의 의사결정 나무를 만들고 투표를 통해 더 나은 모델 선정함



Machine Learning Basics

앙상블 모델(Ensemble Model) – Bagging → Random Forest



Machine Learning Basics

앙상블 모델(Ensemble Model) – Bagging → Random Forest

- 매 실행 시 랜덤하게 관측치와 변수를 선택하므로 실행 결과가 조금씩 변경됨
의사결정나무를 만들 때 데이터의 일부를 복원 추출로 꺼내고 해당 데이터에 대해서만 의사결정나무를 만들
노드 내 데이터를 자식 노드로 나누는 기준을 정할 때 전체 변수가 아니라 일부 변수만 대상으로 하여 가지를 나눌
기준을 찾음.
- 물론 몇몇의 나무들이 과대적합을 보일 순 있지만 다수의 나무를 기반으로 예측하기 때문에 그 영향력이
줄어들게 되어 좋은 일반화 성능을 보임.
- 과대적합을 피하기 위해 임의로 의사결정 나무들을 만들고, 다수의 나무들로부터 분류 결과를 집계하기
때문에 과대적합이 나타나는 나무의 영향력을 줄일 수 있음

Machine Learning Basics

앙상블 모델(Ensemble Model) – Bagging → Random Forest

장점

- 일반 의사결정 나무 알고리즘에 비해 과적합 경향이 적음.
- 매우 높은 차원의 데이터에 대해서도 잘 작동함
- 모델이 매우 유연함으로 전처리를 많이 필요로 하지 않음.
- 다른 분류 알고리즘과 비교했을 때 비교적 좋은 성능을 냄

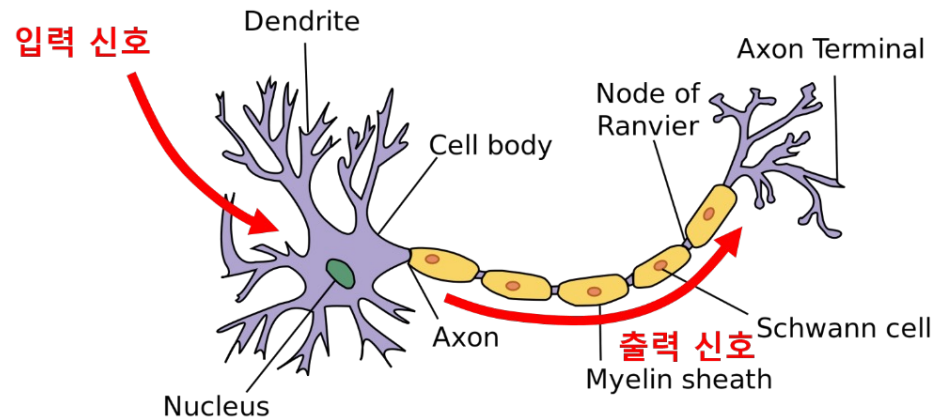
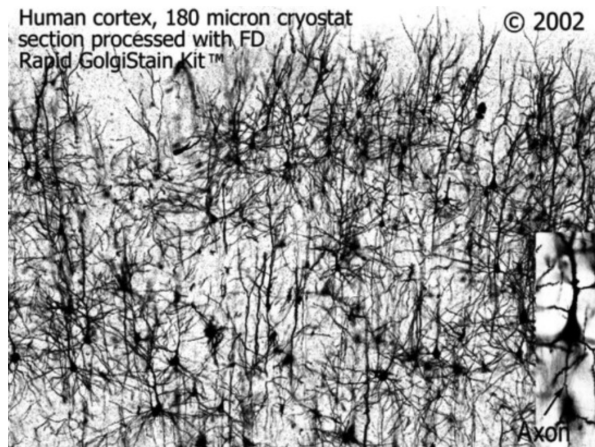
단점

- 많은 컴퓨팅 연산을 필요로 함.
- 모델 아키텍처가 다소 복잡함.

Machine Learning Basics

인공신경망, Artificial Neural Network

- 사람의 신경 세포는 사람 머릿속에 약 1000억개가 있고 각 신경 세포는 다른 1000여 개의 신경 세포와 연결되어 있음.
- 뇌에 어떤 자극이 들어오면 거대한 신경 세포 군집들이 서로 의사소통을 하게 됨.
- 자극을 많이 받으면 받을수록, 해당 전기 신호를 전달하는 신경세포들은 더 촘촘한 망을 이루게 됨 → 학습
- 시냅스에서 뉴런에 전기적 신호를 보내면 뉴런은 이 전기적 신호에 반응하여 다음 뉴런으로 신호를 전달
- 뉴런-시냅스라는 작은 단위가 이루어진 뇌를 통해 인간은 이해하고 고차원적인 사고를 할 수 있음
- Neural Network는 인간의 두뇌를 모방하여 만들어짐



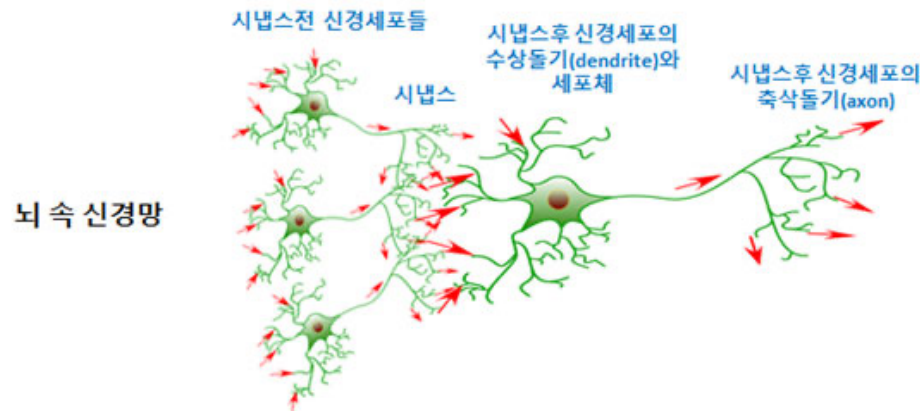
출처: <https://firebasestorage.googleapis.com/v0/b/firescript577a2.appspot.com/o/imgs%2Fapp%2Fhongbae%2FiNXySWN5KK.png?alt=media&token=5333fdd5-612f-40c8-b0f6-81b52d2623ea>

출처: <https://blog.naver.com/infopub/221657609157>

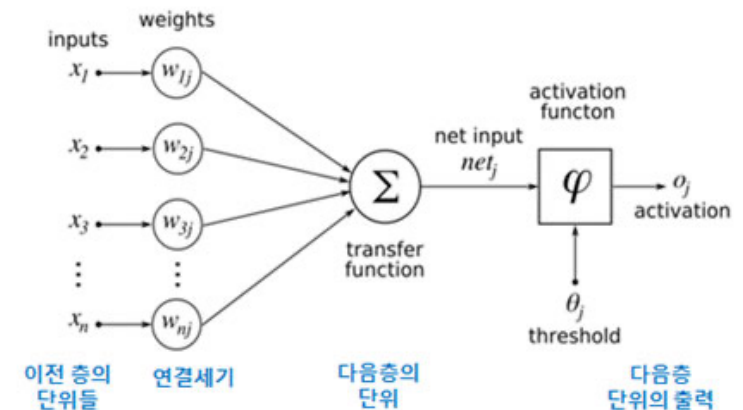
Machine Learning Basics

인공신경망, Artificial Neural Network

- 여러 개의 가지돌기들이 자극을 받아들이고, 종합적으로 받아들이는 결과값들을 다 더한 값이 특정 임계치 이상이면 뉴런이 활성화되고, 그렇지 않으면 활성화 되지 않음
- 축삭돌기에 활성화가 이뤄지면 축삭돌기 말단에서 시냅스를 통해 다음 뉴런으로 신호가 전달되는 과정을 거치게 됨
- 여러 신호를 받아 하나의 신호를 만들어내는 과정을 수식적으로 모형화 함



인공 신경망

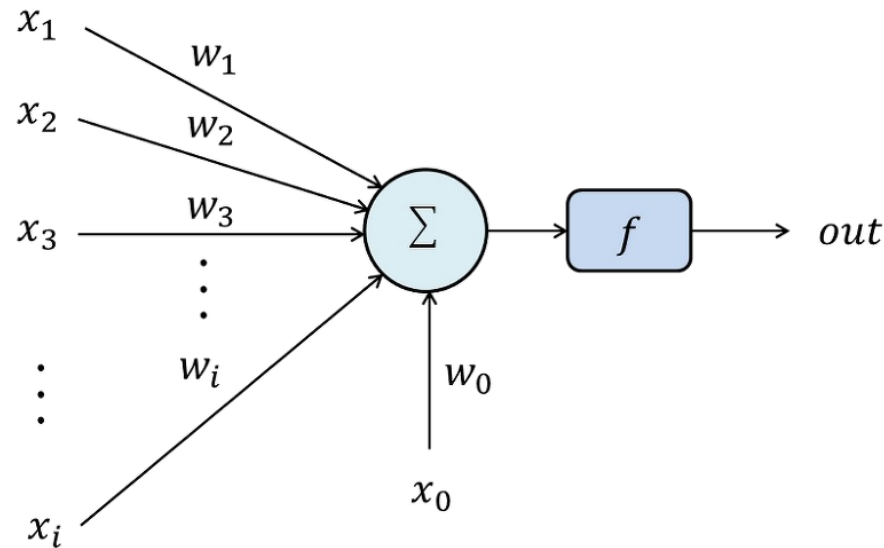
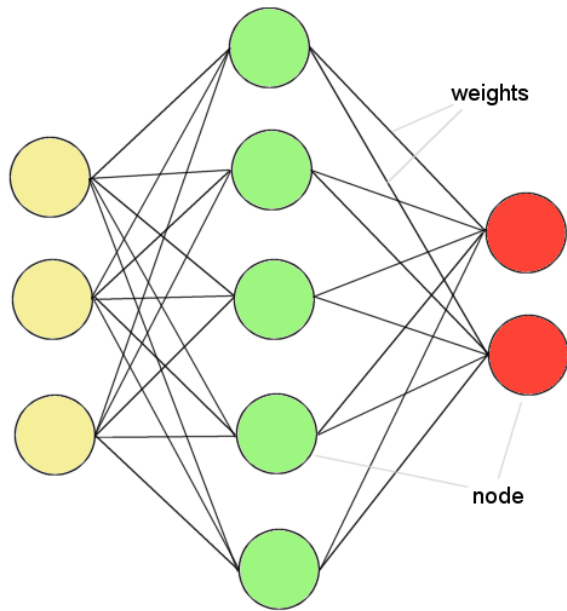


출처: https://itwiki.kr/images/b/be/%EB%87%8C_%EC%86%8D_%EC%8B%A0%EA%B2%BD%EB%A7%9D%EA%B3%BC_%EC%9D%B8%EA%B3%B5_%EC%8B%A0%EA%B2%BD%EB%A7%9D.jpg

Machine Learning Basics

인공신경망, Artificial Neural Network

- 기본 구성 요소: 뉴런 $\rightarrow$ node, 시냅시스 $\rightarrow$ weight)

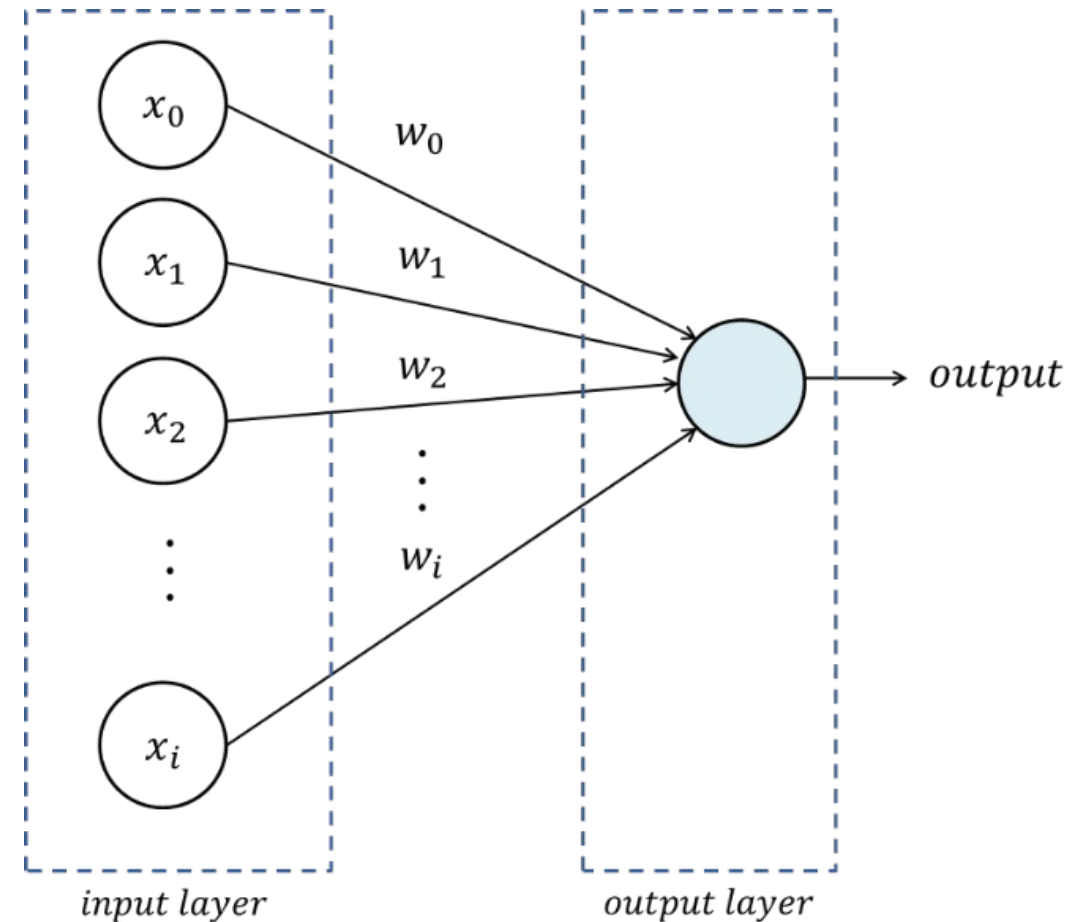


- x 는 입력 벡터의 값
- w 는 가중치
- 바이어스 입력값은 x_0
- 바이어스 기울기는 w_0
- f 는 활성화함수

Machine Learning Basics

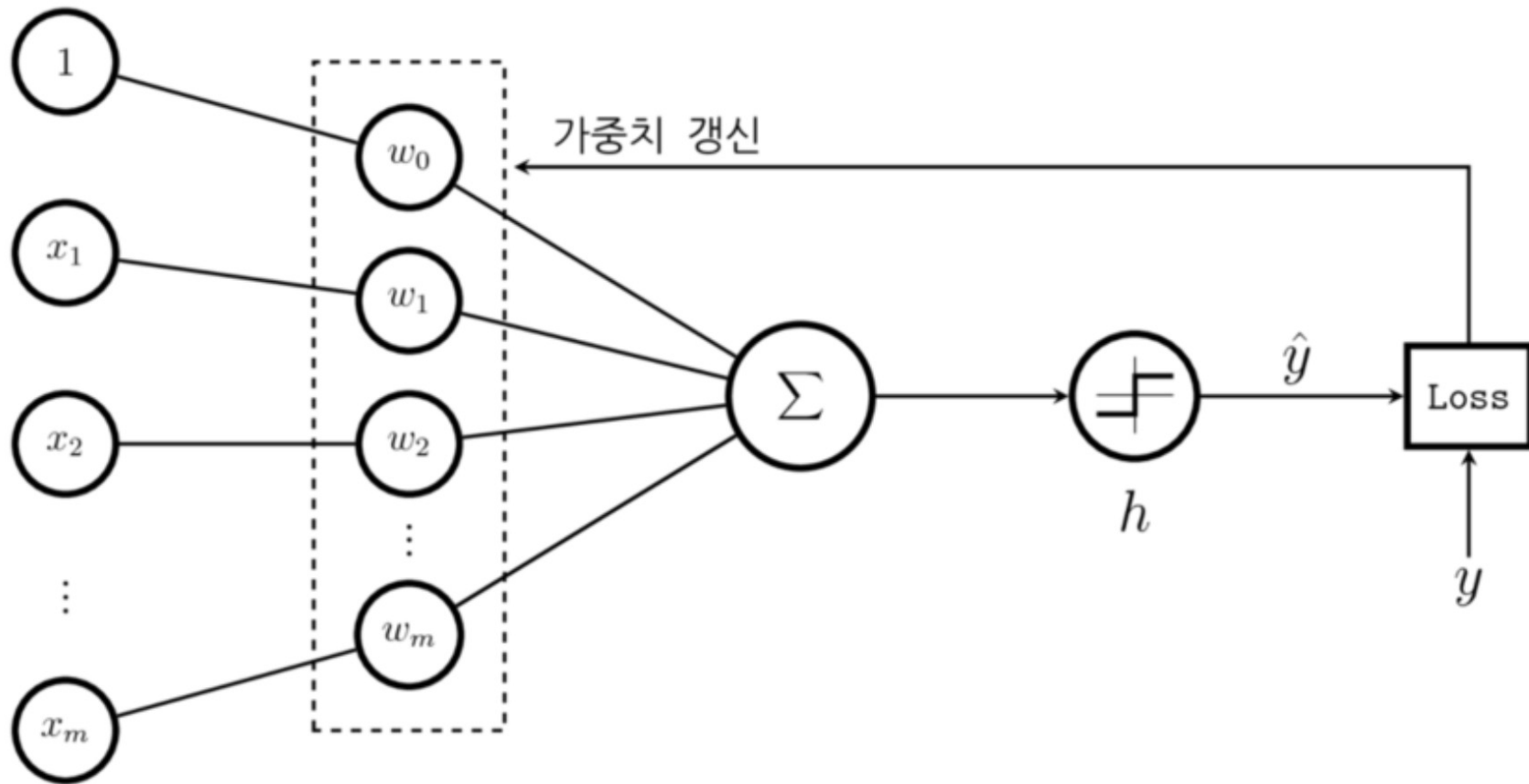
인공신경망, Artificial Neural Network

- 퍼셉트론(Perceptron) : 뉴런을 모방한 것
 1. $x_0 \sim x_n$ 의 신호(Input)을 받아들임.
 2. 다수의 입력을 받았을 때, 퍼셉트론은 각 입력 신호의 세기에 따라 각각 다른 가중치를 부여함.
 3. 가중치와 입력신호들의 합(weighted sum)을 계산
 4. 입력신호의 합이 일정 값을 초과한다면 (활성화 함수(activation function)), 그 결과를 다른 뉴런으로 전달함



Machine Learning Basics

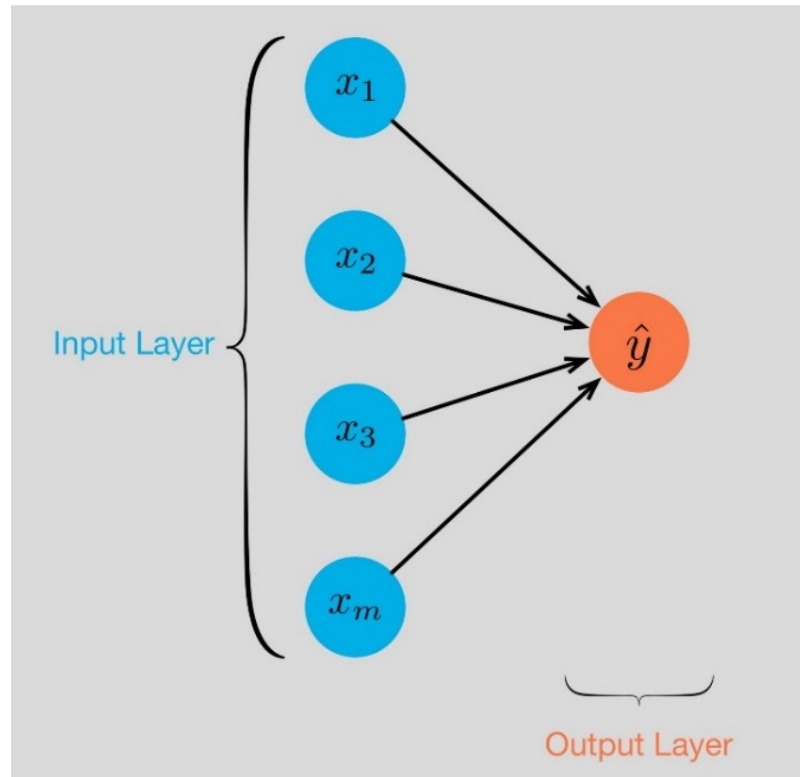
인공신경망, Artificial Neural Network



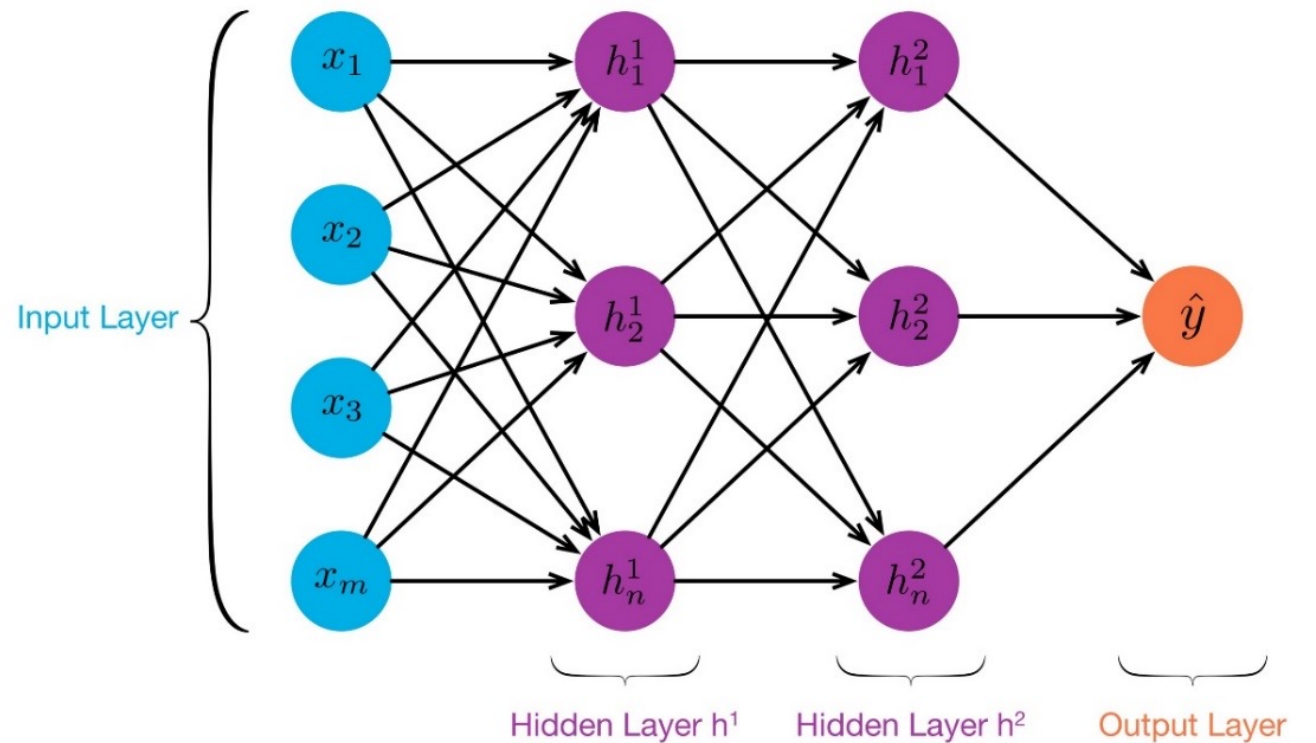
Machine Learning Basics

인공신경망, Artificial Neural Network

Single-layer perceptron



Perceptron with hidden layer



Machine Learning Basics

인공신경망, Artificial Neural Network → Multi-Layer Perceptron (MLP)

- 입력층(Input layer): 입력 값을 받는 층으로 입력 개수만큼 노드가 생성됨
- 은닉층(Hidden layer): 입력층으로 부터 넘어온 데이터를 처리하는 계층, 개수는 1~n개
- 출력층(Output layer): 은닉층에서 처리된 결과를 받아 최종 결과값을 계산해 주는 역할을 수행, 결과 클래스의 개수에 따라 노드 수가 결정됨.

